

Model-Native Computing Architecture

Envisioning Future System Architecture Through the Lens of Computer Architecture

Hai Lin* Shaoxiong Zhan†

Shenzhen International Graduate School, Tsinghua University
Pengcheng Laboratory, Shenzhen, China

May 2026

Abstract

Large language models are undergoing a fundamental transition: from “model technology” to “system technology.” When developers use Codex to write code, Claude Code to manage projects, or AutoGPT to execute multi-step tasks, the engineering challenges that surface—cache reuse efficiency, context capacity management, agent scheduling, and permission control—bear an unmistakable resemblance to classical computer systems problems. A natural analogy emerges: if we treat the LLM as a CPU, the KV cache as a processor cache, the context window as main memory, and the agent framework as an operating system, can eighty years of accumulated wisdom from computer architecture guide us in understanding and building the next generation of model-native computing systems?

This paper pursues that analogy as a *visionary survey*. We first establish a systematic mapping between computer architecture concepts and the emerging stack of model-native systems, then survey the rapidly growing literature across LLM-as-OS, memory management, agent frameworks, tool protocols, multi-agent coordination, cognitive architectures, and safety governance—revealing that each strand addresses a different layer of the same system but lacks a unifying layered model. To fill that gap, we propose the **Intelligent Computing Architecture Model (ICAM)**, which organizes model-native computing into six functional layers, each with clearly defined interface contracts and design axioms. A central tension in the literature—whether the LLM resembles a CPU or an operating system—is resolved by our **dual-plane architecture**: model-native systems simultaneously encompass a probabilistic execution plane (concerned with what *can* be computed) and a deterministic control plane (concerned with what *should* be computed), and prior work has simply been viewing different cross-sections of these two planes.

At the quantitative level, we introduce three design laws: the *Semantic Locality Law*, which characterizes the relationship between KV-cache hit rates and inference speedup; the *Context Budget Law*, which captures the dual constraint of window capacity and attention decay on effective working sets; and the *Agent Speedup Law*, which formalizes the diminishing returns of multi-agent collaboration. Each law is validated against published system-level data. Recent empirical evidence from the Anthropic 2026 agentic practices report further underscores a striking *collaboration paradox*: although multi-agent setups yield diminishing marginal returns, the aggregate economic impact is substantial, with organizations reporting hundreds of thousands of hours saved and significant acceleration of engineering delivery. We close by articulating the boundaries of the analogy, identifying fundamental differences between silicon-era and model-era architectures, and proposing a research roadmap for the coming decade. This paper is a conceptual and survey contribution; it does not present new experimental results.

*ngyygm@outlook.com

†zhansx24@mails.tsinghua.edu.cn

Keywords: LLM systems; computer architecture analogy; Intelligent Computing Architecture Model; quantitative design laws; KV cache; context engineering; agent runtime; dual-plane architecture

1 Introduction: Model-Native Computing Demands an Architecture

Large language models are undergoing a fundamental transition—from a *model technology* to a *system technology*. Starting with GPT-3’s demonstration of in-context learning [?], through the LLaMA family’s catalysis of open-weight ecosystems [?], and onward to the rapid proliferation of tool use [?], retrieval-augmented generation [?], autonomous agents [?], and code generation systems [?, ?], the research frontier has shifted decisively. The central question is no longer “how do we train more capable models?” but rather “how do we organize intelligence into systems that are stable, scalable, and auditable?”

This shift is reshaping engineering practice itself. The engineer’s role is migrating from writing code line by line to orchestrating ensembles of specialized agents, compressing entire development cycles from weeks to hours [?]. Yet many of the bottlenecks that now dominate practice—memory bandwidth walls [?], cache reuse inefficiency [?], context management [?], batch scheduling [?], and sandboxed permission control [?—are not failures of model capability. They are, at their core, *systems problems*.

The impact is already measurable in production. Engineers at Rakuten used Claude Code to autonomously extract activation vectors from the vLLM codebase—12.5 million lines of code—in seven hours, achieving 99.9% numerical accuracy. An Augment Code enterprise customer delivered a project that their CTO had estimated at four to eight months in just two weeks [?]. These are not laboratory demonstrations; they are evidence that the challenges of model-native computing have moved from theoretical possibility to engineering reality.

Given these parallels, a natural question arises: classical computer architecture solved a remarkably similar class of complexity-management problems through a mature stack of layered abstractions. Can that intellectual framework transfer to model-native computing systems?

1.1 A Natural Analogy

The core mapping between the two worlds can be summarized compactly:

Classical Computing	Model-Native Computing	Shared Problem
CPU (general-purpose processor)	LLM inference core	General computation / reasoning
Cache hierarchy (L1/L2/L3)	KV cache	Hot data reuse & memory optimization
Memory & virtual memory	Context window & external memory	Finite-capacity address space management
Operating system	Agent runtime	Scheduling, permissions, resource management
I/O buses (PCIe/USB)	MCP / A2A protocols	Standardized peripheral/tool integration
Applications & users	Agent applications & domain experts	Turning computation into domain solutions

The correspondence runs deeper than surface resemblance. (1) The LLM inference core provides general-purpose semantic understanding and generation, much as a CPU executes a

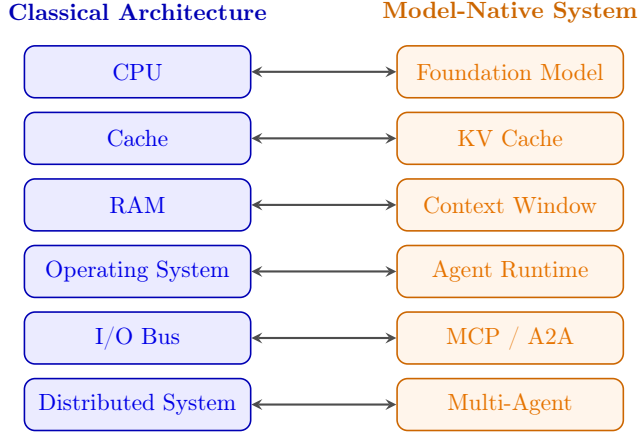


Figure 1: Overview analogy mapping between computer architecture and model-native computing systems

general instruction set—different models (GPT-4, Claude, LLaMA) resemble different microarchitectures that implement the same “instruction set” of reasoning with distinct performance profiles. (2) The *KV cache* reuses previously computed attention key-value pairs during autoregressive decoding to avoid redundant computation, mirroring exactly how a CPU cache exploits temporal locality to reuse hot data. PagedAttention [?] makes this analogy explicit, borrowing the operating system’s paging mechanism to partition KV caches into fixed-size blocks allocated on demand. (3) The context window is strictly bounded (e.g., 128K tokens), yet agents routinely require access to knowledge and interaction histories far exceeding this limit—precisely the problem that motivated virtual memory. MemGPT [?] formalizes this as *virtual context management*, directly analogous to the operating system’s virtual memory abstraction. (4) Agent runtimes such as Codex [?], Claude Code [?], and OpenHands [?] manage task decomposition, tool dispatch, sub-agent coordination, sandbox isolation, and result submission—a responsibility envelope that increasingly resembles an operating system kernel. (5) MCP (Model Context Protocol) [?] defines a standardized interface between agents and external tools, while A2A (Agent-to-Agent Protocol) [?] specifies interoperability contracts among agents. Together they play the role of I/O buses: MCP functions as a vertical bus (analogous to PCIe, connecting an agent to its tools), while A2A functions as a horizontal interconnect (analogous to a network fabric, connecting agents to one another) [?].

In 2023, Andrej Karpathy captured this intuition in a single sentence: “LLMs are not chatbots, they are the kernel process of a new Operating System” [?]. The observation resonated widely precisely because it named something already happening: the engineering problems accumulating around large models look increasingly like classical computer systems problems.

Figure ?? illustrates the overview analogy mapping between computer architecture and model-native computing systems.

1.2 From Intuition to Problem

An analogy, however, is not a theory. “Resembles” does not imply “is.” If the mapping remains at the level of rhetoric, it yields inspirational metaphors but not testable predictions or actionable design principles.

Multiple independent efforts have already explored facets of this analogy from different angles. AIOS treats the LLM as an operating system kernel, building an agent runtime with a scheduler, context manager, and memory manager [?]. MemGPT [?] applies virtual-memory thinking to context management. Mi et al. design agent architectures by drawing on process

models, file systems, and other classical abstractions [?]. Ge et al. articulate the vision of “LLM as OS, Agents as Apps” [?]. L2MAC frames the LLM as a stored-program automatic computer [?]. ArbiterOS defines the LLM as a “Probabilistic CPU” governed by a higher-level governor [?]. AgentOS positions it as a reasoning kernel [?]. MemOS elevates memory itself to a first-class operating-system resource [?]. On the inference engineering side, Mei et al. (2025) survey the emerging discipline of *context engineering*, defining it as the systematic optimization of all information supplied to an LLM [?].

These works each cover one or several layers of the system stack—the OS layer, the memory layer, the agent kernel, the inference layer. Yet they suffer from a fundamental *metaphor conflict*: is the model a CPU, an OS kernel, or a reasoning kernel? More critically, they lack a unified layered model, well-defined inter-layer interfaces, and quantifiable design principles. Moreover, nearly all of them treat the human as a boundary condition of the system rather than as a core component of the interaction loop. The latest industrial evidence, however, tells a different story: the collaboration pattern between human judgment and agent execution—not the agent’s standalone capability—is the primary determinant of system effectiveness. Engineers report using AI for approximately 60% of their work, yet they can *fully delegate* only 0–20% of tasks [?]. This is the collaboration paradox: AI is embedded deeply into the workflow, but meaningful autonomous delegation remains elusive.

The situation echoes the early days of computer architecture. Before the introduction of the Instruction Set Architecture (ISA) as a contract between hardware and software, every new CPU required a complete rewrite of all software, and no improvement at one layer could be leveraged by another. Without an analogous contract for model-native computing, each new system must be built from scratch, and insights remain siloed within individual projects.

1.3 Contributions

Accordingly, this paper makes five contributions.

1. We return to the foundational abstractions of computer architecture, operating systems, and distributed systems to distill an analogy framework suited to the model-native system stack, explicitly delineating where the analogy holds and where it breaks down.
2. We survey the landscape of related work, expose the metaphor conflicts among existing approaches, and propose the **dual-plane architecture** as a unifying resolution—partitioning the model-native stack into a *probabilistic execution plane* (model inference and generation) and a *deterministic control plane* (system scheduling and control), with clearly defined interfaces between the two.
3. Building on this foundation, we introduce the **Intelligent Computing Architecture Model (ICAM)**—a six-layer framework with inter-layer interface contracts and six design axioms.
4. We propose three **quantitative laws** for model-native computing, aiming to serve the same foundational role that Amdahl’s Law and the Roofline model play in classical computer architecture, and we validate them against published data.
5. We survey current open-source implementations and industrial practice, and we chart a research roadmap for the field.

2 Background: The Classical Computing Stack and the Model-Native System Stack

This section first reviews the layered logic of classical computer architecture and the quantitative design principles that underpin it, then surveys the nascent stratification currently taking shape in large language model (LLM) systems. Together, these two perspectives lay the groundwork for the analogy framework developed in subsequent sections.

2.1 Layered Abstractions in Classical Computer Architecture

The layered organization of a classical computing system is not an ad hoc design choice but a general engineering strategy for managing complexity [?]. The central idea is that each layer depends only on the *interface* exposed by the layer below, never on its *implementation*. This separation allows any layer to be independently optimized—or even replaced—without disrupting the rest of the stack. We now review the core concepts of each layer in turn.

2.1.1 Instruction Set Architecture (ISA): The Contract Between Hardware and Software

The *Instruction Set Architecture* (ISA) specifies the machine behavior visible to software: the available instructions (arithmetic, memory access, branching, jumping), the register file, how exceptions are raised and handled, and how the address space is organized. The ISA serves as a stable contract—as long as the processor honors this contract, software above it runs correctly, regardless of the microarchitectural details hidden below. Intel x86, ARM, and RISC-V embody three distinct design philosophies [?, ?]: x86 prioritizes backward compatibility (code written four decades ago still executes), ARM optimizes for energy efficiency (a decisive advantage in mobile devices), and RISC-V embraces openness and modularity (users may define custom instruction extensions). Despite their differences, all three serve the same purpose: providing a stable programming target for the software stack.

2.1.2 Microarchitecture: Multiple Implementations Under One Contract

The *microarchitecture* determines how a given ISA is realized in silicon and, consequently, what performance a processor actually delivers. It answers questions such as: how many clock cycles does an instruction require? How many instructions can execute simultaneously? What is the penalty for a mispredicted branch? Concretely, a *pipeline* decomposes instruction execution into stages—fetch, decode, execute, memory access, write-back—so that different instructions can overlap across stages. A *superscalar* design dispatches multiple instructions per cycle to independent execution units. A *branch predictor* uses historical patterns to speculate on the outcome of conditional branches, thereby avoiding pipeline bubbles. Both Intel’s Core microarchitecture and AMD’s Zen microarchitecture implement the same x86 ISA yet deliver different performance characteristics, and this difference is entirely transparent to software.

2.1.3 Cache Hierarchy: Exploiting Locality to Bridge the Processor–Memory Gap

The *cache hierarchy* addresses a fundamental speed mismatch between processors and main memory. A modern CPU core can complete an operation in roughly one nanosecond, whereas a DRAM access requires approximately 100 nanoseconds—forcing the processor to stall for two orders of magnitude longer if every data reference must reach main memory. Caches exploit the *principle of locality*: *temporal locality* holds that data accessed recently is likely to be accessed

again soon, while *spatial locality* holds that data at nearby addresses will likely be accessed in the near future [?]. Accordingly, caches store recently and nearby accessed data in faster but smaller SRAM: the L1 cache (typically 32–64 KB, ~ 4 -cycle latency) sits closest to the core, the L2 cache (256 KB–1 MB, ~ 10 cycles) acts as a second-level buffer, and the L3 cache (several to tens of megabytes, ~ 40 cycles) is shared among multiple cores. On a cache miss, data must be fetched from DRAM at a cost of roughly 200 cycles. This tiered design trades capacity for speed at each level, using SRAM’s low latency for hot data and DRAM’s density for the working set at large.

2.1.4 Virtual Memory: The Illusion of Infinite Address Space

Virtual memory, managed by the Memory Management Unit (MMU) and page tables, provides each process with an independent address space that can far exceed physical memory. The core mechanism is *paging*: the virtual address space is divided into fixed-size *pages* (typically 4 KB), physical memory into equally sized *page frames*, and a page table records the mapping from virtual page numbers to physical frame numbers. When a process accesses a virtual page not yet resident in physical memory, a *page fault* is raised; the operating system loads the page from disk into a free frame, updates the mapping, and resumes the process [?]. Every process thus believes it possesses a complete, private address space, while the operating system transparently manages allocation and reclamation of the underlying physical resource. Virtual memory also enforces isolation: each process maintains its own page table and cannot address another process’s memory.

2.1.5 Operating System: Scheduling, Protection, and Resource Management

The *operating system* (OS) is the first software layer above hardware. It manages compute resources and provides uniform abstractions to applications. Its core responsibilities include: process scheduling (deciding which process runs next, e.g., Linux’s Completely Fair Scheduler [?]), memory management (allocating and reclaiming physical page frames), I/O management (providing uniform file and device interfaces), access control (ensuring processes access only authorized resources), and concurrency support (semaphores, locks, condition variables) [?, ?]. The OS design principle is simple: expose clean abstractions upward—processes, files, sockets—while managing complex hardware downward—CPU cores, memory modules, disks, network interfaces. Applications invoke system calls such as `open()`, `read()`, and `fork()` without needing to know whether the storage device is an SSD or an HDD, or whether the network link runs at 1 GbE or 10 GbE.

2.1.6 Distributed Systems: Scaling Beyond a Single Node

Distributed systems extend the computing stack across multiple nodes organized into a logically unified whole. The fundamental challenge is maintaining correctness and availability when individual nodes may fail, network latencies are variable, and data must be replicated [?]. The Raft consensus algorithm [?] enables a cluster of nodes to agree on the next command to append to a shared log, tolerating crashes of a minority of participants. Google Spanner [?] provides a globally consistent, distributed database service across multiple data centers worldwide. The CAP theorem [?] formalizes an inherent trade-off: during a network partition, a system cannot simultaneously guarantee both consistency (C) and availability (A); a design choice must be made.

2.1.7 Quantitative Design Principles

What elevates computer architecture from craft to science is a set of **quantitative principles** that make performance predictable and compositional.

Amdahl’s Law. Amdahl’s Law establishes a theoretical upper bound on the speedup achievable through parallelization:

$$S = \frac{1}{(1 - f) + f/p} \quad (1)$$

where f is the fraction of the workload that can be parallelized, p is the number of processors, and S is the resulting speedup. For instance, if 80% of a task is parallelizable ($f = 0.8$) and four processors are available ($p = 4$), the maximum speedup is $S = 1/(0.2 + 0.8/4) = 2.5\times$, far below the ideal $4\times$. Even as $p \rightarrow \infty$, speedup asymptotically approaches $1/(1 - f) = 5\times$. The key insight is that *the serial fraction caps the speedup ceiling*. Malekar and Zand (2024) adapted Amdahl’s Law to the LLM setting, proposing an analytical framework for LLM throughput [?].

The Roofline Model. The Roofline model visualizes the performance constraints of an operator on a given hardware platform [?]. The attainable performance is bounded by two lines: a horizontal line representing peak compute throughput (FLOP/s) and a sloped line representing memory bandwidth multiplied by arithmetic intensity (Bandwidth $\times$ Arithmetic Intensity). Their intersection is called the *ridge point*. Operators with arithmetic intensity below the ridge point are *memory-bandwidth-bound*; those above it are *compute-bound*. Yuan et al. systematically applied the Roofline model to LLM inference, revealing how different optimization strategies trade off bandwidth and compute utilization [?].

Locality Principle. Temporal and spatial locality directly inform cache design: temporal locality (recently accessed data will be re-accessed) motivates cache residency, while spatial locality (nearby addresses will be accessed soon) motivates cache lines (typically 64 bytes), hardware prefetching, and replacement policies such as LRU [?].

Taken together, these quantitative principles have enabled the computing stack to scale from thousands of floating-point operations per second to over 10^{18} operations per second across eight decades—all without requiring programmers to understand the physical details of every layer.

2.2 Emerging Layers of the LLM System Stack

LLM systems are currently undergoing a stratification process that closely mirrors the classical computing stack. We survey each layer from bottom to top, highlighting key developments and the system-level abstractions they implicitly introduce.

2.2.1 Model Layer: Evolution of the Inference Core

On the modeling front, the Transformer architecture [?] remains the dominant backbone. Its self-attention mechanism captures long-range dependencies by computing pairwise interactions across all positions in a sequence. Open-weight models continue to evolve along several axes: RoPE (Rotary Position Embedding) [?] improves positional representations, enabling models to handle longer sequences more effectively; Mixture-of-Experts (MoE) [?, ?] scales model capacity through sparse activation (activating only a subset of parameters per inference), avoiding a proportional increase in computation; long-context extensions [?, ?] push the effective context window from thousands of tokens to hundreds of thousands or even millions. In parallel, selective

state-space models such as Mamba [?] open a linear-complexity sequence modeling paradigm that no longer relies on self-attention.

2.2.2 Inference Layer: Systematic Serving Optimizations

On the inference front, a growing body of work has transformed LLM serving from a single forward pass into a systems engineering problem. FlashAttention [?, ?] reformulates attention computation as an I/O-aware problem: by tiling the computation to reduce accesses to high-bandwidth memory (HBM), it lowers attention memory complexity from $O(N^2)$ to $O(N)$. vLLM’s PagedAttention [?] models KV cache management as a paging problem, dividing the KV cache into fixed-size “pages” that are allocated and reclaimed on demand, thereby eliminating the waste of pre-allocating maximum memory per request. ORCA [?] introduces continuous batching at the iteration level, allowing new requests to join a running batch during the decode phase of existing requests. DistServe [?] decouples the prefill phase (processing the input prompt) from the decode phase (generating tokens one at a time) onto separate GPUs, avoiding resource contention between the two. Sarathi-Serve [?] further refines the throughput–latency trade-off. A unifying theme across these systems is the adaptation of classical operating systems techniques—paging, batch scheduling, resource isolation—to LLM inference engines.

2.2.3 Memory Layer: Reconciling Finite Windows with Persistent State

At the memory layer, the core challenge is that the context window is finite (e.g., 128K tokens), whereas agents often need access to knowledge and interaction histories that far exceed this limit. Multiple technical directions are being explored in parallel. Long-context extensions [?, ?] attempt to enlarge the window directly, but face quadratic growth in computation cost with sequence length. Retrieval-Augmented Generation (RAG) [?] retrieves relevant passages from an external knowledge base at inference time and injects them into the context—analogous to demand paging in an operating system. MemGPT [?] explicitly introduces *virtual context management*, partitioning context into a working context (analogous to main memory) and an external storage (analogous to disk), and automatically moving information between the two. LongMem [?] and Generative Agents [?] explore alternative forms of long-term memory. LongMemEval [?] establishes a benchmark for evaluating memory in long-term interactions. Letta [?] places stateful agents at the center of its design, enabling agents to retain memory across sessions. MemOS [?] goes further by treating memory as a first-class operating-system-level resource, unifying management of textual memory, activation memory, and other representational forms. MemoryOS [?], presented at EMNLP 2025, proposes a memory operating system tailored to AI agents. *Context engineering* [?] has emerged as a nascent discipline that systematizes this space. It is defined as the engineering practice of systematically designing, selecting, and optimizing all information presented to an LLM, encompassing retrieval augmentation, memory management, tool-use scaffolding, and multi-turn dialogue optimization.

2.2.4 Agent Layer: From Single-Turn Inference to Complex Runtimes

At the agent layer, LLMs have evolved beyond single-turn inference engines into controllers within complex runtime environments. ReAct [?] interleaves reasoning with acting, enabling a model to solve multi-step tasks in a “think–act–observe” loop. Toolformer [?] teaches models to invoke external tools—calculators, search engines—autonomously. SWE-agent [?] and OpenHands [?] are autonomous agents for software engineering that can navigate code repositories, write patches, and run test suites. Codex [?] and Claude Code [?] are production-grade coding agents equipped with OS-kernel-like capabilities: sub-agent scheduling [?, ?], sandbox isolation [?, ?], and permission control [?, ?]. Multi-agent collaboration frameworks such as

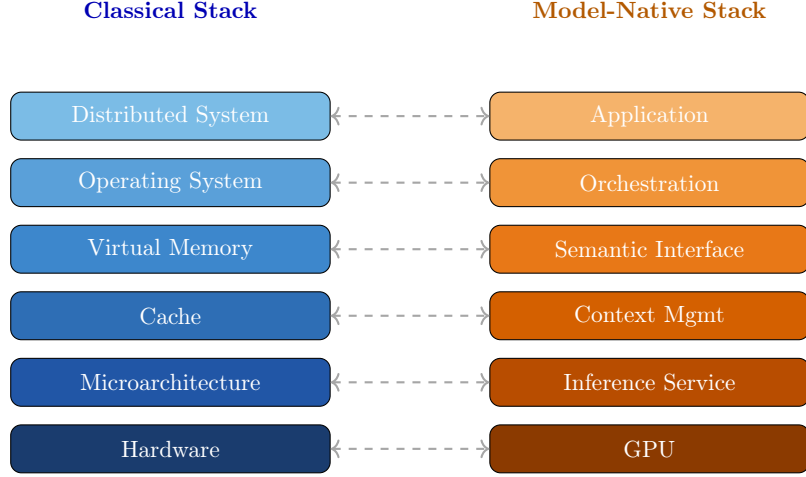


Figure 2: Layer comparison: classical computing stack vs model-native computing stack

AutoGen [?] and MetaGPT [?] further organize ensembles of specialized agents into cooperative networks.

2.3 The Gap: No Unified System Language

Despite rapid progress across each layer, the LLM systems landscape lacks a unified system language—a shared vocabulary and abstraction framework that connects optimizations across layers. Without such a framework, research at each layer tends to fragment into isolated collections of engineering tricks. Consider, for example, *prefix caching* (reusing cached key–value pairs across shared prompt prefixes), *chunked prefill* (breaking long prefill computations into schedulable chunks), *sub-agents* (concurrently executing child agents), *MCP servers* (standardized tool interfaces via the Model Context Protocol), and *persistent memory* (durable storage for agent state). These concepts appear to belong to entirely different research directions, yet from an architectural standpoint they correspond to cache reuse, scheduling granularity, concurrent execution, I/O interfaces, and tiered storage—all well-studied abstractions in classical systems.

This observation motivates the architectural analogy that runs through this paper. The analogy does not claim that LLM systems and classical systems are “fundamentally the same.” Rather, it places independently developed engineering techniques back into a larger, unified design space, making it possible to compose, predict, and systematically design cross-layer optimizations.

Figure ?? compares the layered structures of the classical computing stack and the model-native computing stack.

3 Related Work

The central thesis of this paper is that the analogy framework of *computer architecture* can be used to envision the full layered design of future model-native computing systems. This thesis did not emerge in a vacuum—by 2025, a substantial body of work had independently touched upon key insights such as “LLM systems need OS-style abstractions,” “agents require memory management,” and “tool invocation demands standardized protocols.” This section surveys the most relevant literature along functional themes, progressively revealing the contribution and limitation of each line of work, and ultimately positioning the unique contribution of this paper.

3.1 LLM-as-OS / Agent Kernel

The earliest systematic articulation of the “LLM as operating system” analogy is due to Ge et al. (2023), who proposed “LLM as OS, Agents as Apps” [?]. Their central insight can be summarized by a system mapping diagram: the LLM corresponds to the OS kernel (responsible for core inference and scheduling), the *context window* corresponds to main memory, external storage corresponds to the file system, hardware tools correspond to peripherals, software tools correspond to programming libraries, and user prompts correspond to user commands. In other words, they reconceived the LLM not as a “chatbot” but as a “kernel process”—a metaphor later popularized by Karpathy on social media [?]. Within the analogy framework of this paper, the work of Ge et al. directly corresponds to the embryonic notion of the Layer 2 Agent OS.

Mei et al.’s AIOS [?] advanced this line of thinking from an analogy to a runnable system implementation. The core design of AIOS decouples responsibilities that were previously entangled with application logic in the agent runtime and places them into a dedicated “kernel,” including: (1) a scheduler managing concurrent execution of multiple agents; (2) context management distributing the limited context window across agents; (3) memory management handling reads and writes of short-term and long-term memory; (4) storage management governing files and persistent data; and (5) access control governing agent permissions over tools and data. Experimental results show that this decoupled design yields up to $2.1\times$ speedup. In our analogy framework, AIOS realizes an early Agent OS prototype whose kernel responsibilities—process management, memory management, and file system modules—map directly onto those of a traditional operating system.

AgentOS [?] went further by defining the LLM as a *reasoning kernel* constrained by “structured operating system logic”—that is, the LLM is not a free-form inference engine but a controlled reasoning core that must operate within the structural framework of OS logic. This perspective emphasizes that the LLM’s reasoning capabilities must be orchestrated and constrained by structured system logic, rather than allowing the model to generate without restraint.

By 2025–2026, this line of research advanced from academic prototypes toward real desktop and mobile scenarios. Microsoft’s UFO² [?] organizes a HostAgent (for understanding user intent), an AppAgent (for operating specific applications), a GUI-API hybrid action layer, and virtual-desktop parallel operations into a “Desktop AgentOS.” Its innovation lies in simultaneously supporting both GUI screenshot comprehension and native API calls as interaction modalities. Aura [?], targeting mobile scenarios, adopts a hub-and-spoke topology: a privileged System Agent parses user intent, while multiple sandboxed App Agents each handle a specific domain. Its design philosophy is “security first”—inter-agent communication must be mediated through the System Agent, analogous to how inter-process communication in a traditional OS must pass through the kernel.

Relationship to this paper. The works above each define a role for the LLM within the system—OS kernel, reasoning kernel, or Agent Kernel—but they have not converged on a unified layered model. The ICAM model proposed in Section ?? consolidates these disparate “kernel” concepts into a six-layer architecture and explicitly positions the LLM as Layer 1 (the probabilistic compute core) while the Agent OS occupies Layer 2 as an independent abstraction—the two should not be conflated.

3.2 Memory Hierarchy and Virtual Context Management

Large language models have a finite *context window*—for example, GPT-4 Turbo supports 128K tokens, Claude 3 supports 200K tokens, and Gemini 1.5 Pro reaches up to 1M tokens [?]. Yet regardless of how large the model’s window grows, the usable capacity remains bounded, which is structurally identical to the “finite physical memory” problem in classical computing. A series

of works has therefore borrowed **memory management** techniques from operating systems to expand the effective context available to agents.

MemGPT [?] (later evolved into the Letta platform) is the foundational work in this direction. Rather than simply attaching a retrieval-augmented generation (RAG) [?] pipeline to look up external knowledge, MemGPT draws directly on the **virtual memory management** mechanism of a traditional OS: it treats the limited context window as “main memory,” external databases as “disk,” and employs an OS-like memory manager that automatically pages information fragments in and out between the context window and external storage, giving the agent the illusion of unbounded context. MemGPT’s control flow also explicitly employs *interrupts*—when information must be retrieved from external storage, an interrupt transfers control to the memory manager. This design maps directly onto the “virtual memory” concept in our analogy.

MemOS [?] elevates the problem to a more systematic plane. It unifies three granularities of memory into manageable system resources: (1) *plaintext memory*—external knowledge stored as text; (2) *activation-based memory*—internal hidden-layer representations of the model (analogous to CPU registers); and (3) *parameter-level memory*—knowledge encoded in model parameters (analogous to firmware/ROM). MemOS uses the “MemCube” as an encapsulation unit, bundling content with provenance and versioning metadata, and attempts to build a migration bridge between retrieval and parameter learning. This work corresponds to the “multi-level storage hierarchy from KV cache to parameter storage” in our analogy.

MemoryOS [?] applies OS memory management principles directly to the agent memory system. HiAgent [?] (published at ACL 2025) proposes “hierarchical working memory management”—using subgoals as memory chunking units to organize long-horizon task memory into a hierarchy, analogous to multi-level page tables in an operating system. A-MEM [?] introduces the concept of *agentic memory*: memory is not a passively stored record but a dynamic resource that agents can autonomously organize, recombine, and update. LongMemEval [?] further demonstrates that long-term memory is not simply “storing more chat logs,” but a complex systems engineering challenge involving information extraction, temporal reasoning, knowledge updating, and the ability to refuse to answer.

Relationship to this paper. Research on memory subsystems has become highly systematic, yet it is typically discussed in isolation from the agent runtime and I/O layers, lacking a full-stack perspective. More importantly, existing work has not explicitly identified the *KV cache* (the key-value cache maintained during model inference) as the counterpart of “hardware cache” in the analogy—whereas this paper argues that the KV cache is precisely the key hardware abstraction bridging the probabilistic compute core and the semantic memory hierarchy. We elaborate on this correspondence in Section ??.

3.3 Agent Frameworks and Runtimes

If the preceding two themes define the “kernel” and “memory,” respectively, this section focuses on how programs execute on top of that kernel—that is, the execution frameworks and runtime environments for agents.

Foundational capabilities: interleaving reasoning and action. ReAct [?] is the foundational work on agent execution paradigms. Prior to ReAct, reasoning (e.g., Chain-of-Thought) and acting (e.g., invoking tools) were separate, sequential steps for LLMs. ReAct demonstrated that **interleaving** reasoning and action within a single loop—“think one step, act one step, observe the result, think again”—significantly improves task completion quality. This is analogous to the fetch-decode-execute cycle in traditional computing, where the CPU executes an instruction, accesses memory, and checks a condition in tight alternation.

Toolformer [?] addressed another foundational question: enabling the model to learn **when**

to call APIs on its own. By automatically inserting API call examples into training data, Toolformer empowers the model to autonomously decide during generation whether it needs to invoke a particular tool—analogous to the *system call* mechanism in an operating system, where a user program initiates a request to the kernel.

The rise of coding agents. The most striking agent advances in 2024–2026 have concentrated in software engineering. OpenAI’s Codex CLI [?], released in April 2025, is a terminal-based agent programming assistant. It can read code repositories, edit files, execute commands, and operate safely within a sandboxed environment [?]. In early 2026, Codex CLI introduced a subagent feature [?]¹—the main agent can spawn specialized subagents that execute subtasks in parallel, each with its own independent context window and sandbox. This directly mirrors the `fork()` model in traditional operating systems, where a parent process creates child processes. Codex also supports custom instructions via AGENTS.md files [?] and reusable capability modules through Skills [?].

Anthropic’s Claude Code [?] is another representative agent runtime. Like Codex, it is a terminal-native coding agent capable of editing files, executing shell commands, and connecting to external services. An independent research study systematically analyzed Claude Code’s system architecture [?] and found that only approximately 1.6% of its codebase constitutes AI decision logic, while the remaining 98.4% is *operational harness*—including safety controls, tool orchestration, and context management. Claude Code likewise supports subagents [?] and Skills-based extensions [?], and provides project-level memory through CLAUDE.md files [?].

Notably, the temporal span of agent execution is expanding from minutes to hours or even days. Rakuten engineers tested Claude Code on the vLLM project (12.5 million lines of code) to perform activation vector extraction; the agent ran autonomously for 7 hours and achieved 99.9% numerical accuracy [?]. This trend means that agent runtimes must handle state persistence, checkpoint recovery, and cross-step resource management far beyond what current interactive sessions require.

OpenHands [?] (formerly OpenDevin) is an open-source general-purpose agent platform published at ICLR 2025. It achieves approximately 70–74% issue-resolution rates on the SWE-bench Verified benchmark [?] and supports multiple LLM backends. SWE-agent [?] focuses specifically on software engineering tasks, achieving comparable performance with a more streamlined Agent-Computer Interface (ACI) design.

Relationship to this paper. These agent frameworks already exhibit operating-system-like characteristics in their architecture: Codex’s subagents correspond to process management, Claude Code’s CLAUDE.md corresponds to configuration and environment management, and OpenHands’ ACI corresponds to the system call interface. However, they remain independently designed “application-layer” tools lacking a unified layered abstraction. The ICAM model proposed in this paper provides a unifying theoretical framework that maps their functionality onto well-defined architectural layers.

3.4 Tool Calling and I/O Protocols (MCP, A2A)

Agents must interact with the external world—querying databases, calling APIs, manipulating file systems. In classical computing, this requirement corresponds to the I/O subsystem (input/output devices and buses). The key development over the past two years is that I/O abstractions are moving from ad hoc tool-calling conventions toward **standardized protocols**.

HuggingGPT [?] is an early representative work: it places the LLM in a “controller” position, tasking it with parsing user requests, selecting appropriate expert models, coordinating execution, and aggregating results. This is analogous to the device driver management layer in an operating system, where the OS uniformly manages calls to different devices (models). Go-

rilla [?] focuses on a more specific problem: when API documentation is frequently updated, how does the model maintain call accuracy? This corresponds to the version-compatibility problem for device drivers.

MCP: the model-to-system I/O bus. In November 2024, Anthropic introduced the Model Context Protocol (MCP) [?], defined as an open standard for connecting AI applications with external data sources, tools, and workflows. MCP adopts a client-server architecture: an MCP Client runs on the AI application side, while an MCP Server encapsulates a specific tool or data source. Through a unified JSON-RPC protocol, any MCP-compatible AI application can connect to any tool that provides an MCP Server—Anthropic has likened MCP to “USB-C for AI.” By 2026, the MCP ecosystem has expanded rapidly, with Claude Code [?] and numerous third-party tools all supporting MCP connections.

A2A: the agent-to-agent interconnect protocol. In April 2025, Google announced the Agent-to-Agent Protocol (A2A) [?] at Cloud Next ’25, explicitly positioned as a complementary open protocol to MCP. A2A addresses not the “model-to-tool” connection problem but the “agent-to-agent” collaboration problem—supporting capability discovery, task lifecycle management, long-running tasks, and multimodal coordination. A2A was donated to the Linux Foundation in 2025 [?], becoming an industry standard candidate. A2A employs a peer-to-peer client-server model in which every agent can act as both a requesting client and a responding server.

Relationship to this paper. Taken together, MCP plays the role of a model-to-system I/O bus (analogous to USB/PCIe), while A2A serves as an agent-to-agent interconnect protocol (analogous to the TCP/IP networking stack). In the ICAM model proposed in this paper, these correspond to Layer 4 (I/O and tool layer) and Layer 5 (interconnect and protocol layer), respectively. No existing work has yet integrated both protocols into a unified layered abstraction or examined their interactions with context management, kernel scheduling, and other layers at the architectural level.

3.5 Multi-Agent Collaboration as Distributed Systems

When multiple agents collaborate, the system exhibits characteristics of distributed computing—a development that parallels the historical evolution from standalone machines to distributed systems.

AutoGen [?] (Microsoft Research, 2023) is the foundational framework for multi-agent systems. Its core design treats *multi-agent conversation* as universal infrastructure: agents in different roles coordinate tasks through natural language dialogue. AutoGen has evolved to v0.4 and has been integrated into the Microsoft Agent Framework, supporting scalable, production-grade multi-agent systems.

MetaGPT [?] (ICLR 2024 Oral) introduced a critical design idea: explicitly encoding the **Standard Operating Procedure** (SOP) from human organizations into multi-agent collaboration. Specifically, MetaGPT simulates a virtual software company: a product manager writes requirements, an architect designs the system, engineers write code, and a QA engineer tests—each role is an agent, and the SOP defines the workflow and information flow among them. This is analogous to *workflow orchestration* in distributed systems, constraining unordered agent interactions into an ordered assembly line.

This pipeline pattern has been validated at industrial scale. Fountain’s Copilot system employs a central orchestrator agent that coordinates three specialized sub-agents for candidate screening, document generation, and sentiment analysis. This hierarchical multi-agent deployment achieved 50% faster candidate screening, 40% faster onboarding, and 2× candidate conversion rates, compressing the full staffing center configuration cycle from over one week to

under 72 hours [?]. This hub-and-spoke topology contrasts sharply with MetaGPT’s pipeline topology and AutoGen’s peer-to-peer topology, suggesting that coordination topology itself is a key design variable influencing multi-agent system performance [?, ?].

The Internet of Agents (IoA) [?] (published at ICLR 2025) envisions the agent ecosystem from an “internet” rather than single-system perspective. Its core contribution is an agent integration protocol supporting dynamic team formation, collaboration, and distributed execution among heterogeneous agents. IoA draws design inspiration directly from the Internet: just as the Internet connects computers from different manufacturers running different operating systems, IoA aims to connect agents built with different frameworks by different organizations. This has also spurred IETF-level standardization efforts (IoA Protocol Internet Draft).

Relationship to this paper. Multi-agent systems already exhibit “distributed system” morphology at the object-of-study level: AutoGen corresponds to message-passing mechanisms, MetaGPT to workflow orchestration, and IoA to network interconnect protocols. However, most work remains at the “framework and protocol” layer without integrating with individual agent memory management, kernel scheduling, I/O, and access control into a unified architecture. The ICAM model maps multi-agent collaboration onto Layer 5 (interconnect and protocol layer) and Layer 6 (application layer), and discusses cross-layer interaction design principles.

3.6 Cognitive Architectures and Security Governance

The preceding subsections have surveyed related work from a functional systems perspective (kernel, memory, runtime, I/O, multi-agent). This section turns to two higher-level themes: what cognitive architecture should agent systems follow, and how should their security be governed?

Cognitive architectures. CoALA (Cognitive Architectures for Language Agents) [?] provides a theoretical framework grounded in cognitive science. CoALA organizes language agents into three core components: (1) *modular memory*—divided into short-term working memory and long-term memory (semantic, episodic, and procedural); (2) *structured action space*—internal actions (reasoning, memory retrieval) and external actions (tool invocation, environment interaction); and (3) a *generalized decision-making process*—a plan–execute–observe loop. The value of CoALA lies in providing the theoretical anchor for “why this system is not merely a software stack assembly, but a genuine cognitive architecture.” L2MAC [?] (ICLR 2024) explicitly frames its architecture as the “first practical LLM-based stored-program automatic computer (von Neumann architecture),” comprising an instruction register, file storage, control unit, and independent LLM agent modules. Mi et al. [?] organize their LLM agent survey from a “computer systems perspective,” explicitly inspired by the von Neumann architecture: they decompose agents into perception, cognition, memory, tool, and action modules, observe that the context window is analogous to main memory and databases to disk, and point out that agents currently lack a “cache module.”

Security governance. ArbiterOS [?] identifies a fundamental problem: we are using the mental model of **deterministic software engineering** to direct what are **inherently probabilistic processors**. This contradiction demands the introduction of a governance layer at the system architecture level. ArbiterOS accordingly proposes the “Agentic Computer” mental model—redefining the LLM as a *Probabilistic CPU* and designing governance-layer concepts including a governor, a formal instruction set, and a hardware abstraction layer (HAL). Within our analogy framework, ArbiterOS’s core insight—that models are probabilistic and require a deterministic governance layer—is one of the direct inspirations for the *dual-plane architecture* (probabilistic execution plane + deterministic control plane) proposed in this paper.

CaMeL [?] (Google DeepMind, 2025) applies operating-system capability-security principles to LLM agents from a safety perspective. Its core design philosophy is “Don’t execute data”—strictly separating trusted instructions from untrusted data, extracting control flow and data

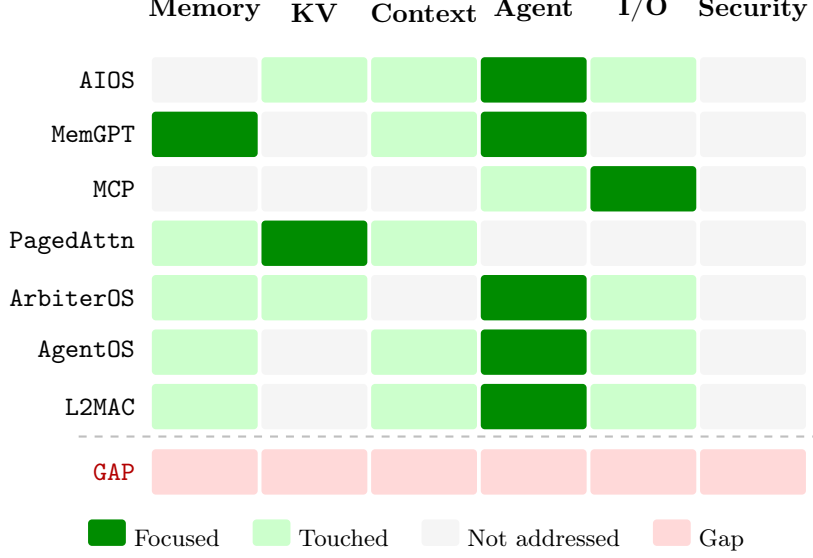


Figure 3: Coverage of system layers by existing works

flow and enforcing separate security policies on each to defend against prompt injection attacks. IronClaw [?] (2026) explicitly advocates protecting AI agents in the same way an operating system protects its processes—migrating OS security mechanisms such as privilege separation, sandbox isolation, and audit logging into the agent system.

Relationship to this paper. The cognitive architecture line provides theoretical foundations, while the security governance line supplies engineering constraints. Yet these two threads remain unintegrated—cognitive architectures rarely address security, and security governance rarely considers cognitive structure. The dual-plane architecture proposed in this paper attempts to unify both: the probabilistic execution plane carries cognitive capabilities, while the deterministic control plane enforces security governance, and the two interact through well-defined interface contracts.

3.7 Common Boundaries and Unsolved Problems

The six themes surveyed above collectively approach the vision of a “model-native computing architecture,” yet four critical problems remain unsolved in a unified manner.

Figure ?? shows the coverage of each system layer by existing work.

Problem 1: What role does the LLM actually play? Existing work exhibits a fundamental divergence in positioning the LLM within the system. Ge et al. and AIOS place the LLM at the core of the OS [?, ?]—the LLM *is* the kernel; ArbiterOS demotes it to a “Probabilistic CPU” governed by the kernel [?]-the LLM is a managed processor; AgentOS defines it as a reasoning kernel constrained by structured OS logic [?]-the LLM is a controlled reasoning core. These three positions lead to fundamentally different system designs. The ICAM model proposed in this paper offers a unified answer: the LLM is Layer 1 (probabilistic compute core / CPU), and the Agent OS is Layer 2 (operating system kernel); they are distinct abstraction layers and should not be conflated. The LLM provides compute capability, while the Agent OS provides management and scheduling—just as the CPU and the OS kernel are two independent yet tightly cooperating layers in classical computer architecture.

Problem 2: No full-stack layered model exists. No mainstream work simultaneously places the probabilistic compute core, semantic memory hierarchy, tool and peripheral bus, Agent OS/kernel, interconnect protocols, governance and security layer, programming model and in-

struction semantics, and evaluation metrics within a single architectural framework. L2MAC [?] focuses on the stored-program computer model; Mi et al. [?] provide a von Neumann-inspired survey framework, but each covers only one facet of the architecture. Memory management work does not discuss I/O protocols; agent frameworks do not discuss KV cache; security governance does not discuss multi-agent collaboration—each line of work makes genuine contributions, but their boundaries are clearly delineated.

Problem 3: The probabilistic–deterministic tension has not been codified architecturally. ArbiterOS [?] correctly identifies the tension between probabilistic processors and deterministic governance, but it proposes only a conceptual “governance-first” paradigm. CaMeL [?] and IronClaw [?] provide concrete security mechanisms, but they are not embedded within a complete layered architecture. This paper *codifies this tension architecturally* through the dual-plane architecture: the probabilistic execution plane carries the LLM’s inference capabilities, while the deterministic control plane enforces OS-level management and security responsibilities, with the two planes cooperating through explicit interface protocols.

These first three unsolved problems define the contribution space of this paper: providing a unified, layered, dual-plane architectural model that integrates all six threads within a single master diagram.

Problem 4: Structural constraints of human–agent collaboration have not been modeled. Existing work treats the human as a boundary condition of the system (Layer 6 user), but the latest industrial evidence reveals a “collaboration paradox”: engineers use AI for approximately 60% of their work, yet can fully delegate only 0–20% of their tasks [?]. Engineers tend to delegate tasks that are “easy to verify or low-risk” while retaining those that are “conceptually difficult or depend on design decisions.” This structural constraint suggests that the human–agent interaction loop should become an explicit component of the architecture, rather than an external assumption.

4 Analogy Framework: Intelligent Systems as Layered Computers

4.1 Principles of Analogy

The analogy framework underlying this paper is summarized in Table ?? . Before presenting the detailed mapping, we establish three principles to ensure that the analogy serves as a rigorous analytical instrument rather than a decorative metaphor.

Principle 1: The analogy must correspond to genuine engineering mechanisms. The observation “KV cache resembles a processor cache” holds not because the names sound alike, but because both serve the same fundamental goal of exploiting *temporal locality* and *state reuse* [?, ?]. Temporal locality—the empirical regularity that recently accessed data is likely to be accessed again in the near future—governs both a CPU cache line and a recently computed set of KV vectors. Similarly, the observation “PagedAttention resembles virtual memory” is valid because vLLM literally organizes KV blocks into fixed-size pages and maps them through a block table to non-contiguous physical memory, mirroring the mechanism by which an operating system manages virtual pages [?]. In each case, the analogy is grounded in shared engineering structure, not superficial naming.

Principle 2: The analogy must acknowledge its boundaries. A model is not a deterministic CPU: executing `ADD R1, R2` always produces the same result, whereas the same prompt may yield different responses across invocations. Semantic retrieval is not exact addressing: `MOV [0x7fff], EAX` always writes to the same location, while vector similarity search returns results that vary with the embedding model and query phrasing. An agent runtime is not a mature

operating system: the Linux kernel rests on four decades of formal verification and regression testing, whereas the safety mechanisms of current agent frameworks are still evolving rapidly. These “not quite identical” gaps are not weaknesses of the analogy—they define precisely the research space where the most impactful future work lies.

Principle 3: The goal of the analogy is to generate research questions. A productive analogy does more than aid understanding; it surfaces new engineering problems. Which modules require clearly defined interface contracts? Which shared states demand consistency guarantees? Which execution paths require access control and audit logging? The rightmost column of Table ?? captures the research directions that the analogy generates.

Within this framework, a future model-native computing architecture can be understood as follows: **a system that uses the foundation model as its computing core, KV and context hierarchies as its storage layer, an agent kernel as its control layer, tools and the external world as its I/O layer, and serving clusters with shared memory as its distributed substrate.** In Section ??, we unpack each component in detail, proceeding from core to periphery.

4.2 Metaphor Conflict and the Dual-Plane Architecture

Before examining individual components, we must resolve a fundamental conceptual tension. As discussed in Section ??, the existing literature is deeply divided on where the LLM sits in the architectural stack. Ge et al. place the LLM in the position of the OS kernel [?]. AIOS incorporates it alongside resource management, context management, and access control as kernel-level services [?]. AgentOS treats the LLM as a “reasoning kernel” constrained by structured operating system logic [?]. ArbiterOS demotes the LLM to a “Probabilistic CPU” governed by a separate control layer [?].

These positions appear contradictory, but in reality each captures only one facet of the system. We argue that a model-native computing system simultaneously encompasses **two distinct planes**, and that prior work has simply been looking at different cross-sections.

4.2.1 The Probabilistic Execution Plane

The probabilistic execution plane is, at its core, the foundation model’s forward inference process.

Concrete example. When a user submits a code review request to GPT-4, the model’s attention mechanism performs matrix operations over billions of parameters, ultimately producing a token probability distribution from which the response is sampled. The same input may yield different outputs across invocations; the quality of the output depends heavily on how the context is organized; and the model itself holds no persistent state—the results of one conversation do not automatically carry over to the next.

The key characteristics of this plane are:

- *Probabilistic output:* the output follows a probability distribution; identical inputs can produce different results.
- *Context sensitivity:* small changes in prompt formulation can significantly alter output quality.
- *No persistent state:* model weights remain static across requests; all “memory” resides in the tokens within the context window.

KV caches, attention mechanisms, and decoding strategies all belong to the “microarchitecture” of this plane—they optimize the efficiency and quality of the model’s forward computation itself.

4.2.2 The Deterministic Control Plane

The deterministic control plane is the programmatic control layer that wraps around the model—the scheduler, permission enforcer, state machine, logging system, and failure recovery logic.

Concrete example. When Claude Code receives the request “refactor this function,” the deterministic control plane executes the following sequence: check current directory permissions (is the path within the allowed project scope?) → load the project memory file (`CLAUDE.md`) → parse the request and decompose it into subtasks → for each step involving file writes, determine whether user confirmation is required based on the approval policy → upon completion, write a change log to the audit record[?]. Every step in this sequence is executed by deterministic code; its behavior is predictable, auditable, and replayable.

The key characteristics of this plane are:

- *Deterministic execution:* given the same inputs and policy configuration, the control plane always reaches the same decisions.
- *Formal policies:* permission rules and approval gates are explicitly defined in code or configuration files.
- *Auditability:* every decision step is logged and supports post-hoc review.

Codex’s approval policies[?], Claude Code’s read-by-default stance and hook system[?], and CaMeL’s capability-based security mechanisms[?] all belong to this plane.

4.2.3 Why the Separation Is Necessary

Industrial practice has already validated the necessity of this separation. Praetorian’s development platform embeds LLMs within a deterministic orchestration layer, substantially improving agent reliability in autonomous software development[?]. The “deterministic shell, probabilistic core” pattern is emerging as the de facto standard architecture for agent systems[?]. The “Blueprint First, Model Second” framework encodes operational logic as deterministic source-code blueprints, constraining the LLM to operate within well-defined boundaries[?].

Figure ?? illustrates the separation between the probabilistic execution plane and the deterministic control plane.

The core rationale for separation is a clean division of responsibilities:

- The probabilistic plane excels at *understanding and generation*—it answers what *can* be done.
- The deterministic plane excels at *control and assurance*—it governs what *should* be done.
- The probabilistic plane should *not* directly execute side-effectful operations (writing files, calling external APIs); instead, such actions must be mediated through the deterministic plane’s approval and audit channels.

This separation finds strong support in recent empirical evidence. Anthropic’s internal study reveals that engineers use AI for approximately 60% of their work tasks yet feel comfortable fully delegating only 0–20%[?]. This **collaboration paradox** is not a temporary artifact of insufficient model capability; it is a structural property of dual-plane systems. The deterministic control plane—including human judgment transmitted through permission policies and approval workflows—naturally constrains the action space of the probabilistic plane. Engineers tend to delegate tasks that are “easy to verify or low-risk” while retaining tasks that are “conceptually difficult or depend on design decisions.” This is precisely the behavior of a deterministic plane acting as a gate for the probabilistic plane. Without separation—if the model could directly execute any operation—such graduated delegation would be impossible.

The analogy to classical architecture is instructive. In a traditional system, user-mode code cannot access hardware directly; it must issue a system call that traps into the kernel[?]. User

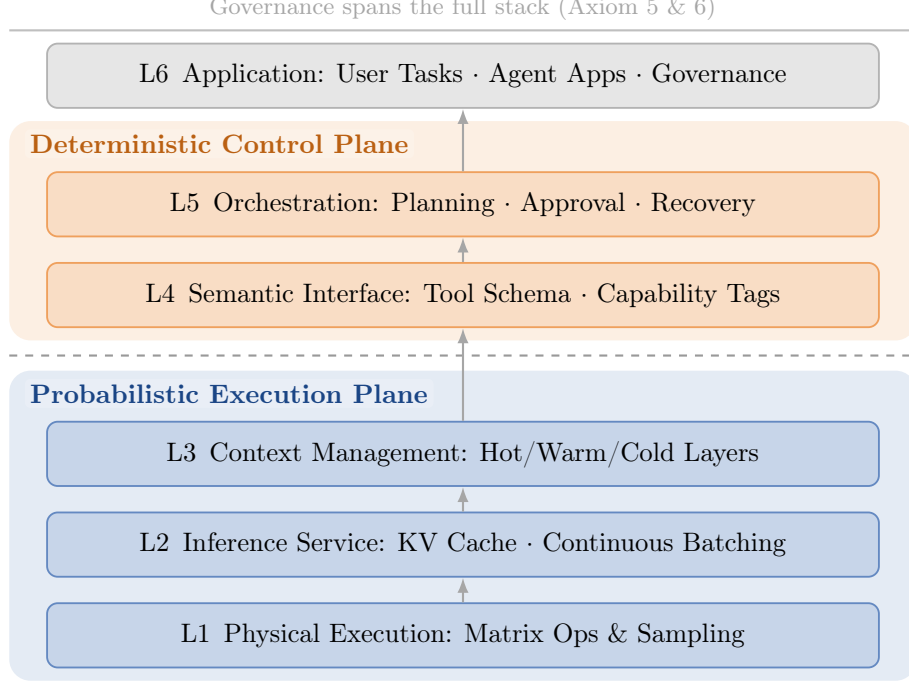


Figure 4: Dual-plane architecture: probabilistic execution plane (blue, L1–L3) and deterministic control plane (orange, L4–L5)

programs (the probabilistic plane) may issue arbitrary requests, but the kernel (the deterministic plane) decides which requests are permitted, at what priority they execute, and how they are logged and recovered.

The metaphor conflict is thereby resolved. When ArbiterOS calls the LLM a “Probabilistic CPU,” it is describing the probabilistic execution plane. When AIOS incorporates the LLM into a “kernel,” it is observing a mixture of the probabilistic plane and partial control logic. When AgentOS proposes a “reasoning kernel,” it is attempting to impose the structure of the control plane onto the probabilistic plane. These positions are not contradictory—they are complementary descriptions of the same dual-plane system, viewed from different vantage points.

5 Key Component Analysis

This section analyzes six key components in order from core to periphery: foundation model (corresponding to the CPU) → KV cache (corresponding to the cache hierarchy) → context management (corresponding to virtual memory) → agent runtime (corresponding to the operating system) → tool bus (corresponding to I/O) → multi-agent collaboration (corresponding to distributed systems). For each component, we follow a uniform four-part structure: (1) the classical computer architecture counterpart, (2) the corresponding construct in LLM-based systems, (3) why the two are analogous at the mechanism level, and (4) the essential differences that the analogy should not obscure.

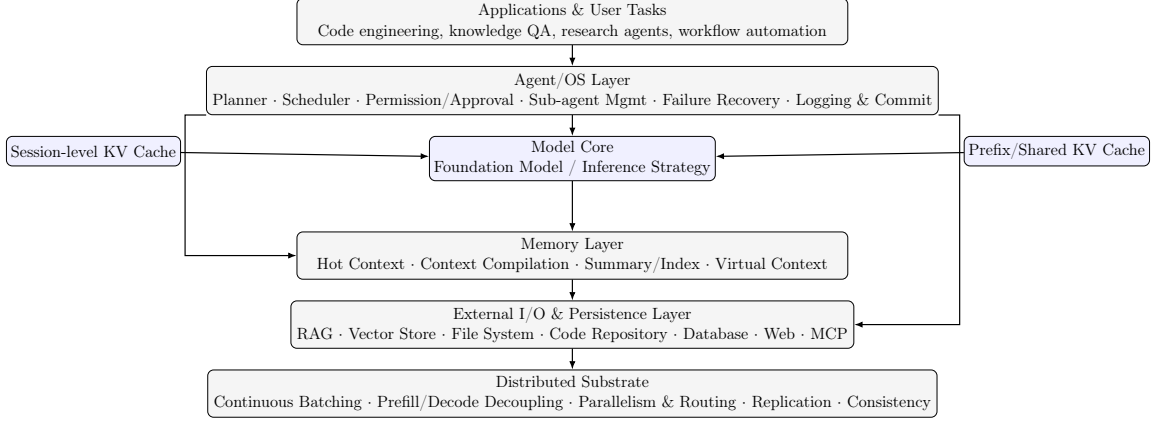


Figure 5: Layered architecture for model-native computing. Top to bottom: application layer, Agent/OS layer, model core with KV cache, memory layer, I/O layer, distributed substrate. Bypass arrows indicate the deterministic control plane’s direct management of memory and I/O layers.

5.1 Foundation Model: Probabilistic Computing Core

5.1.1 The CPU in Classical Computer Architecture

In a classical computer, the CPU (Central Processing Unit) serves as the system’s execution core. Its responsibility can be stated concisely: **execute instructions in sequence, mapping input state to output state**. Internally, the CPU comprises an arithmetic logic unit (ALU) for computation, a register file for holding operands, and a control unit for instruction fetch and decode. The defining property is **determinism**: executing `ADD R1, R2, R3` one million times yields the identical result provided the input registers hold the same values. CPU behavior is precisely specified by the instruction set architecture (ISA); for instance, the RISC-V specification gives a mathematically rigorous description of every instruction’s semantics [?].

5.1.2 The Counterpart in LLM-Based Systems

In a model-native computing system, the foundation model (e.g., GPT-4, Claude, LLaMA) plays the role of a *general-purpose execution core*. Given an input context—comprising a system prompt, user messages, tool results, and control constraints—the model produces the next action or output through a forward pass [?, ?, ?].

Prompts, tool descriptions, skill files, and project memory files (such as `AGENTS.md` or `CLAUDE.md`) can be understood as a form of **soft instruction interface**: they do not alter the model’s physical weights—just as software does not alter the CPU’s circuitry—yet they significantly reshape the model’s execution path [?, ?, ?]. For example, a carefully authored `CLAUDE.md` file can cause the same model to exhibit “Python expert” behavior in one project and “Rust expert” behavior in another—much as loading different programs onto the same CPU yields different behaviors.

SGLang has already moved in this direction: it reformulates complex language-model programs as a collaboration between a structured front-end language and a high-performance runtime, analogous to the transition from assembly language to high-level languages [?, ?].

5.1.3 Why the Analogy Holds

The analogy between a CPU and a foundation model rests on three pillars.

First, generality. A CPU is a general-purpose compute core—the same silicon can run a word processor or a scientific simulation. A foundation model is a general-purpose reasoning core—the same weights can write code, analyze data, translate languages, and plan tasks. In both cases, the substrate is fixed while behavior is customized through “software” (programs versus prompts).

Second, interface abstraction. A CPU exposes operations to upper layers through the ISA (arithmetic, branching, memory access); a model exposes operations through prompt templates and tool schemas (reasoning, generation, tool invocation). Both adhere to the principle of “stable interface, evolvable implementation.” The x86 architecture has persisted for over four decades while its microarchitecture evolved from single-scalar to superscalar out-of-order execution; likewise, the GPT-4 API has remained stable across successive model updates.

Third, microarchitectural optimization space. CPUs benefit from pipelining, branch prediction, and superscalar issue; models benefit from FlashAttention (I/O-aware attention computation [?]), speculative decoding (using a small model to predict a larger model’s output [?]), and continuous batching. Both face the engineering challenge of “improving performance without changing the interface.”

5.1.4 Essential Differences

The crux of the difference is **determinism versus probability**. A CPU executes a formal instruction set in which every instruction has precisely defined, long-term-stable, fully reproducible semantics. A foundation model processes a mixture of natural language, code, and structured schemas, producing outputs drawn from a probability distribution. As Mi et al. have argued, drawing on the evolution of computer architecture to inform agent system design requires candid acknowledgment of this fundamental gap between probabilistic and deterministic execution [?].

Consequently, the future “intelligent ISA” should not be understood as hard-coding prompts as fixed instructions. Rather, the research agenda should prioritize developing more stable interface layers: tool schemas, constrained output formats, reusable skill packages, task-specific DSLs, and explicit context declarations. From an architectural perspective, this means the community should evaluate models not only by parameter count but also by **model programmability**—the degree to which upper layers can reliably and predictably control model behavior.

5.2 KV Cache: Cache Hierarchy for Intelligent Systems

5.2.1 The Cache Hierarchy in Classical Computers

In a classical computer, caches resolve the fundamental tension between fast processors and slow memory [?]. A modern CPU typically has three cache levels: L1 (~ 1 ns, 64 KB) $\rightarrow$ L2 (~ 4 ns, 512 KB) $\rightarrow$ L3 (~ 12 ns, several to tens of MB). Caches exploit **temporal locality**: data accessed recently is likely to be accessed again soon. When the CPU needs data, it checks L1 first, then L2, then L3, and finally fetches from main memory. Cache management revolves around three core questions: *placement policy* (where to store data), *replacement policy* (what to evict when full), and *write policy* (how to propagate modifications).

5.2.2 The Counterpart in LLM-Based Systems

During autoregressive inference, generating each new token requires computing attention between that token’s Query vector and the Key/Value vectors of *all* preceding tokens. Recomputing the full prefix KV at every step would incur $O(n^2)$ cost in sequence length. The KV cache avoids this by storing previously computed Key and Value vectors, reducing each attention step

to an $O(1)$ lookup and bringing overall inference cost from $O(n^2)$ to $O(n)$. This rationale mirrors that of classical caching: **trade capacity (GPU memory) for latency and bandwidth advantages** [?, ?].

A KV cache hierarchy is emerging:

- **L1—Session-level KV cache:** KV vectors within a single session; lowest latency, capacity bounded by GPU memory.
- **L2—Prefix/shared cache:** KV blocks shared across requests for common system prompts or codebase indices, analogous to read-only code pages shared among processes in a CPU [?, ?, ?].
- **L3—Semantic cache:** Reusable tool-call results for similar queries, or reusable indices for identical repository structures—“semantic-level” cache hits.

When vLLM introduced PagedAttention—organizing KV vectors into fixed-size blocks mapped through a block table to non-contiguous physical memory—the analogy ceased to be merely heuristic and became a direct borrowing of virtual-memory page management [?].

5.2.3 Why the Analogy Holds

The similarity between KV caches and CPU caches operates at the level of engineering mechanisms.

First, both exploit temporal locality. CPU caches retain recently accessed cache lines because “data used recently will be used again soon”; KV caches retain recently computed KV vectors because “generating the next token requires attending to all prior tokens.” Both keep frequently reused data in fast storage to avoid expensive recomputation or memory accesses.

Second, both face capacity pressure and replacement decisions. L1 caches have limited capacity and employ LRU-style replacement policies; KV caches face analogous GPU memory pressure. H2O identifies “heavy-hitter” tokens (those receiving the highest attention weights) and implements intelligent eviction [?], analogous to identifying “hot data” in CPU caches. StreamingLLM discovered that initial tokens receive disproportionately high attention (the “attention sink” phenomenon) and achieved unlimited-length streaming inference by pinning these sink tokens [?]*—directly analogous to cache pinning in CPU caches.*

Third, both encounter bandwidth bottlenecks. Gholami et al. observe that LLM inference during the decode phase is fundamentally memory-bandwidth-bound, not compute-bound [?]. Yuan et al. systematically characterized this behavior using the Roofline model [?]. Li et al. organized KV cache management strategies into three tiers—token-level (which KV entries participate in attention), model-level (architectural optimizations such as Grouped-Query Attention (GQA) and Multi-Query Attention (MQA)), and system-level (memory management and scheduling) [?]*—mirroring the multi-level approach taken in CPU cache optimization.*

Fourth, compression is a shared optimization direction. KVQuant achieved 8× compression [?]; KIVI applied asymmetric 2-bit quantization with nearly lossless accuracy and dramatically reduced memory footprint [?]. This parallels data compression in CPU caches (e.g., IBM MXT) to increase effective capacity.

5.2.4 Essential Differences

First, hit criteria differ fundamentally. CPU cache hits are **Boolean**: the address matches or it does not. In the KV cache, prefix caching does involve exact matching (identical prefix token sequences are reused), but upper-level semantic cache hits are **continuous-valued**: a similarity score exceeding a threshold is treated as a hit, and different thresholds yield different precision–recall trade-offs. Classical cache hit-rate models therefore do not transfer directly to

semantic caching.

Second, invalidation mechanisms differ. CPU cache coherence is ensured by hardware protocols such as MESI—when data is modified, the corresponding cache lines are marked invalid. KV and semantic cache “invalidation” is far more subtle: tool version changes, code repository commits, and stale indices can all produce hits that return outdated answers—a phenomenon we might call *semantic staleness*. Recent work has begun exploring learned eviction policies [?] and dependency-aware invalidation mechanisms [?] to address this challenge.

Third, the capacity–latency trade-off is more severe. In the CPU cache hierarchy, latency increases gradually from L1 to L2 to L3 to main memory on a nanosecond scale. In the KV cache hierarchy, the latency jump from GPU memory to CPU memory to disk or network spans milliseconds, with no smooth intermediate gradient. Future cache management will need to co-design traditional cache policies with provenance tracking, TTLs, and hash-based versioning.

5.3 Context Management: Virtual Memory with Semantics

5.3.1 Virtual Memory in Classical Computers

Virtual memory resolves a core tension: “programs need far more address space than physical memory provides.” The operating system creates the illusion of abundant space through **paging**: each process receives its own virtual address space (e.g., 48-bit addressing, 256 TB), while physical memory may be only 16 GB. When a process accesses a virtual page not yet mapped to physical memory, a page fault occurs, and the OS loads the required page from disk. The central advantage is **demand paging**—only data actually accessed occupies physical memory, and the entire process is transparent to the application.

Key components of virtual memory management include: page tables (mapping virtual to physical addresses), TLBs (caches that accelerate address translation), page replacement policies (deciding which pages to evict to disk), and write-back policies (determining when dirty pages are flushed to disk) [?].

5.3.2 The Counterpart in LLM-Based Systems

The context window is to a foundation model what main memory is to a process: a high-bandwidth, low-latency, but capacity-limited **working set**. Anthropic defines the context window directly as the working memory available to the model during generation.

Context window capacity is expanding rapidly: Gemini 1.5 Pro supports up to 10 million multimodal tokens [?]; Ring Attention distributes sequences across multiple devices to achieve nearly unlimited context length [?]; positional encoding methods such as RoPE, YaRN, and LongRoPE aim to extend the addressable history range [?, ?, ?].

Yet a growing body of evidence shows that “declared support length” and “effective working-set capacity” are not the same thing. RULER, LongBench v2, and LOFT provide more realistic stress tests for long-context capability [?, ?, ?]. A machine may advertise 256 TB of virtual address space while having only 16 GB of physical memory; true performance depends on whether the working set fits in physical memory at all times.

The more principled design, therefore, should not aim to “stuff everything into context at once,” but rather adopt **tiered management** in the spirit of virtual memory.

An emerging tiered scheme:

- **Hot memory (within the context window):** Complete information for the current task, analogous to active pages in physical memory.

- **Warm memory (summaries and indices):** Compressed historical context, analogous to pages swapped out but retaining summary metadata.
- **Cold memory (external knowledge bases):** Persistent vector stores and document repositories, analogous to data on disk.

MemGPT formalizes this idea as *virtual context management*, explicitly drawing the LLM-as-operating-system analogy and paging information between the context window (“main memory”) and external storage (“disk”) [?]. LongMem decouples long-term memory from the backbone model entirely [?]. RAG functions as an **external page-in mechanism**—when the agent encounters a “page fault,” it retrieves relevant content from a knowledge base into the context window via semantic search [?]. MemoryOS applies OS memory management principles directly to agent memory systems [?]. MemOS proposes a comprehensive memory operating system architecture [?]. HiAgent, inspired by human problem-solving processes, uses sub-goals as memory blocks to hierarchically manage agent working memory [?]. A-MEM enables agents to organize memory structures autonomously, without predefined schemas [?].

5.3.3 Why the Analogy Holds

The analogy between context management and virtual memory holds at the **mechanism level**.

First, both create the illusion of abundant space. Virtual memory convinces each process that it has a contiguous, large address space, when physical memory is far smaller. Virtual context management convinces each agent that it possesses unlimited memory, when the actual context window spans only 128K–1M tokens. Both realize this illusion through **demand loading**: only information needed right now occupies “physical memory” (the context window); the rest resides in “disk” (external knowledge stores).

Second, both face an “address translation” problem. Virtual memory uses page tables to map virtual addresses to physical addresses; virtual context uses semantic retrieval to map “information needs” to specific knowledge entries. The former is an exact match (page number to physical page number); the latter is an approximate match (query intent to top- k relevant documents).

Third, both require “paging policies.” When physical memory runs low, the OS must decide which pages to swap out (LRU, Clock, and related policies); when the context window fills up, the agent must decide which information to evict (summarization, selective forgetting). This is precisely the function of the “context compiler”—the system must determine which state is loaded verbatim, which as a summary, which retains only an index entry, and which is discarded entirely [?].

5.3.4 Essential Differences

The key distinction: virtual memory is lossless; context management is lossy. OS paging is fully transparent to the application—pages swapped in and out retain every byte unchanged. Agent context management, by contrast, involves **semantic compression**: summarization inevitably loses detail, and selection inevitably omits information. “Address translation” is also imprecise: vector similarity search returns the “most relevant” results, not the “exact match.”

Semantic virtual memory can therefore never be a lossless paged system. It is better understood as “paged memory augmented with semantic summarization and evidence backfill”—information may be distorted after compression, but provenance chains enable verification against original data. Research from JetBrains suggests that effective context management depends not on “cramming in more tokens” but on “reducing noise and preserving signal” [?]
further validating the necessity of lossy compression in this regime.

5.4 Agent Runtime: The Intelligent Operating System

5.4.1 The Operating System in Classical Computers

In a classical computer, the operating system is the system software that “manages hardware resources, isolates applications, and provides unified service interfaces” [?]. Its core responsibilities can be summarized in three abstractions: **virtualization** (giving each process the illusion of exclusive CPU and memory access), **concurrency** (enabling multiple processes to run simultaneously and safely), and **persistence** (reliably storing and managing files).

Concretely, the OS handles: responding to system calls (file reads and writes), scheduling processes (which runs first and for how long), isolating processes (one crash does not affect another), managing memory (allocation, reclamation, paging), enforcing permissions (who can access which resources), handling I/O (interacting with peripheral devices), and maintaining logs (auditing and fault recovery).

5.4.2 The Counterpart in LLM-Based Systems

If the foundation model is the CPU, then the agent framework increasingly resembles the OS. An agent runtime must:

- **Parse objectives:** Decompose user requests into executable subtasks (analogous to program loading and parsing).
- **Decide whether to plan:** Route simple requests directly and decompose complex ones into multi-step plans (analogous to fast-path versus slow-path scheduling).
- **Load context:** Populate the context window with system prompts, project memory, and conversation history (analogous to loading a process memory image).
- **Execute tool calls:** Invoke external tools via protocols such as MCP (analogous to system calls).
- **Approve dangerous actions:** Perform permission checks on side-effecting operations such as file writes or code execution (analogous to capability-based access control).
- **Spawn sub-agents concurrently:** Dispatch subtasks to independent child agents (analogous to fork).
- **Merge intermediate results:** Aggregate sub-agent outputs into a final answer (analogous to inter-process communication and result aggregation).
- **Persist state:** Write important state to memory files or knowledge bases (analogous to filesystem writes).
- **Recover from failure:** Retry or roll back on errors to maintain task consistency (analogous to transaction recovery).

This analogy has already been articulated in the literature. Ge et al. proposed the vision of “LLM as OS, Agents as Apps” [?]. Mei et al. built AIOS, a full LLM agent operating system kernel architecture [?]. Mi et al. systematically applied the evolutionary experience of computer architecture to agent system design [?]. Karpathy argued in 2023 that “LLMs are not chatbots; they are the kernel process of a new operating system” [?]¹—the filesystem becomes a vector database, the browser becomes internet search. This vision is becoming an economic reality: an Augment Code enterprise customer completed a project in two weeks that their CTO had estimated would take four to eight months [?], demonstrating that effective L3 context management combined with L5 orchestration can compress developer onboarding and project delivery from weeks to days.

Industrial practice is evolving in the same direction. Codex provides subagents, skills, and sandbox-based control surfaces [?]; Claude Code offers hooks, MCP integration, memory files, and read-only approval flows [?].

5.4.3 Why the Analogy Holds

Among all the analogies in this paper, the correspondence between the agent runtime and the OS is the deepest, because it operates at the level of **architectural responsibility**.

First, both are resource managers. The OS manages CPU time slices, memory pages, disk space, and network bandwidth; the agent runtime manages model inference quota, context window space, tool-call rate limits, and sub-agent count. Both must answer: “How should limited resources be allocated among competing demands?”

Second, both provide isolation and security. The OS isolates processes through address spaces—one process crash does not affect others [?]; the agent runtime isolates sub-agents through sandboxes—one sub-agent’s erroneous operation does not compromise the main task [?, ?]. Both face the same access-control question: “Who is allowed to do what?”

Third, both define standard interfaces. The OS standardizes application–hardware interaction through system calls (the POSIX interface); the agent runtime standardizes model–tool interaction through protocols such as MCP [?].

5.4.4 Essential Differences

First, agent “process” behavior is unpredictable. A traditional OS schedules processes whose behavior is defined by deterministic code; the OS can accurately predict execution time and resource requirements. An agent’s behavior is determined by probabilistic reasoning—the same task may follow entirely different execution paths across runs, succeeding once and falling into an infinite loop the next. This poses a fundamental challenge for scheduling and resource management.

Second, shared state is far more fragile. When multiple processes share memory, consistency is ensured by hardware cache coherence protocols (e.g., MESI) and software locks—precise and verifiable. When multiple agents share semantic state (e.g., “an understanding of the same document”), consistency can only be maintained through natural-language communication or external synchronization mechanisms—ambiguous and error-prone.

Third, interface specifications are not rigorous. Every system call in the POSIX interface has precisely defined semantics, parameter types, error codes, and boundary conditions. An agent’s tool-call interface can describe parameters via JSON Schema, but the “semantic behavior” of a tool—what it returns under which circumstances—is far from the level of precision expected of a system call.

The greatest value of this analogy lies in reframing many agent problems as OS problems:

- Does multi-agent collaboration resemble multithreading or multiprocessing? [?]
- Should tool invocations be treated as system calls?
- Is an MCP server analogous to a device driver or bus protocol? [?]
- Why do dangerous commands require capability-based permissions? [?]
- How can long-running tasks recover via logs and interrupts? [?]

Security: a particularly compelling analogy. In the security domain, this correspondence is especially profound. CaMeL applies operating system capability-security principles to LLM agents, creating a protection layer independent of the LLM itself [?]. IronClaw argues for protecting AI agents the way an operating system protects its processes [?]. Systems security researchers have begun using the term “agentic computing” to define the foundational problems of agent security [?]. These efforts underscore a critical insight: **security boundaries are primarily a “runtime structure” problem and only secondarily a “prompt” problem**—just as OS security depends first on the permission model and isolation mechanisms, and

only then on application code quality.

5.5 Tool Bus and Agent Interconnection

5.5.1 The I/O System in Classical Computers

In a classical computer, the I/O system connects the CPU and memory to the external world. Its evolution is highly instructive: In the **early phase** (1960s–1970s), each peripheral required a dedicated interface and driver—graphics cards had their own bus, hard drives their own controller, printers their own parallel port. In the **standardization phase** (1990s–2000s), PCI/PCIe unified the motherboard-level bus and USB unified peripheral connectivity—hardware vendors needed only to follow a common protocol for plug-and-play operation. In the **networking phase** (2000s–present), the TCP/IP protocol stack unified network communication—any device could interoperate through standard protocols.

The central lesson of this history is: **a standardized bus protocol is a prerequisite for ecosystem prosperity.**

5.5.2 The Counterpart in LLM-Based Systems

In model-native computing, the I/O layer is undergoing a similar transition from point-to-point connections to a standardized bus.

Early phase (method-level): ReAct interleaved reasoning with acting [?]; Toolformer taught models when to call APIs [?]; HuggingGPT placed an LLM in a “controller” role to orchestrate heterogeneous models [?]; Gorilla focused on the accuracy and documentation-refresh resilience of API calls [?]. These efforts correspond to the “direct device manipulation” phase of early computing: each tool required a bespoke interface and calling convention, with no unified protocol standard.

Standardization phase (protocol-level): MCP (Model Context Protocol) standardizes the connection between AI applications and external data sources, tools, and workflows [?], playing a role analogous to PCIe or USB in traditional computing—it defines unified discovery, connection, invocation, and error-handling mechanisms so that new tools can be “plug-and-play.” Google’s A2A (Agent-to-Agent) protocol targets agent-to-agent interoperation [?], supporting capability discovery, task lifecycle management, and multimodal collaboration—analogueous to TCP/IP in the networking stack, enabling agents from different vendors to discover and cooperate with one another.

Taken together, MCP serves as a **vertical bus** (agent→tool) while A2A serves as a **horizontal interconnect** (agent→agent). As one survey observes, MCP, A2A, and related protocols such as ACP are forming the infrastructure layer for agent interoperability [?].

5.5.3 Why the Analogy Holds

First, both solve the “interface diversity” problem. Before PCIe unified motherboard-level device interfaces, every device needed a dedicated driver; before MCP unified tool interfaces, every tool required bespoke API adapter code. Both reduce system integration costs by “defining a standard protocol that hides implementation differences.”

Second, both exhibit a layered protocol stack. Traditional I/O spans the physical layer (electrical signaling), the link layer (PCIe transaction layer), the protocol layer (NVMe/SCSI), and the application layer (filesystem). Agent I/O follows a similar stack: transport layer (HTTP/SSE), protocol layer (MCP/A2A message formats), semantic layer (tool

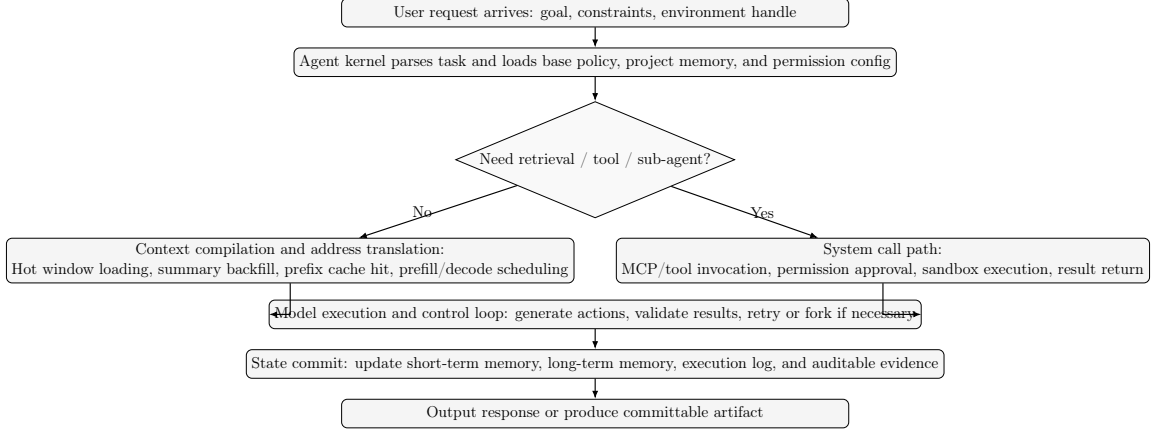


Figure 6: Control flow for a typical model-native computing request. Diamonds represent deterministic control plane decisions; rectangles alternate between probabilistic execution and deterministic control.

schemas and agent capability descriptions), and application layer (concrete tool invocations or agent collaborations).

Third, both face discovery and negotiation challenges. A USB device, upon insertion, must be “enumerated”—the host queries the device type, capabilities, and driver requirements. An MCP server, upon connection, similarly undergoes “capability discovery”—the agent queries which tools the server supports, along with each tool’s parameter schema and side-effect description.

5.5.4 Essential Differences

First, invocation determinacy differs. A traditional system call such as `read(fd, buffer, size)` is fully predictable—identical parameters yield identical results. Tool invocations, by contrast, may be non-deterministic (search engine results change over time), asynchronous (long-running tasks require callbacks), and side-effecting (sending email, executing code).

Second, error handling is far more complex. PCIe devices either respond correctly or return a well-defined error code; MCP tools may return semantically ambiguous errors (“the result is uncertain”) or—worse—results that appear correct but are in fact wrong. This compels the agent I/O layer to introduce **verification and redundancy** mechanisms: critical operations require result validation, analogous to quorum reads in distributed systems.

5.6 Multi-Agent Collaboration: Distributed Systems with Semantics

5.6.1 Distributed Systems in Classical Computing

When a single computer lacks the compute power to complete a task, multiple machines must cooperate over a network—this is the domain of distributed systems. The core challenges include: **task decomposition and scheduling** (breaking a large task into smaller ones and assigning them to different nodes), **state synchronization and consistency** (how multiple nodes agree on shared state), **fault detection and recovery** (how the system continues when a node crashes), **deadlock avoidance** (breaking cyclic waits among nodes), and **load balancing** (keeping all nodes busy rather than overloading some while others idle) [?].

5.6.2 The Counterpart in LLM-Based Systems

When multiple agents collaborate on a complex task, the system exhibits characteristics of distributed computing.

AutoGen treats multi-agent conversation as general infrastructure [?]¹—multiple agents cooperate through message passing, much like distributed nodes. MetaGPT uses pipeline-style Standard Operating Procedures to make multi-agent collaboration explicitly procedural [?]²—analogous to industrial-pipeline task decomposition, where each agent handles one stage (requirements analysis → design → coding → testing). Internet of Agents imagines the agent ecosystem as an “internet” rather than a single machine [?]³—emphasizing agent integration protocols, dynamic team formation, and distributed environments.

From an architectural perspective, clear correspondences emerge: a single agent maps to a single process, multi-agent collaboration to multithreading or multiprocessing, and a cross-node agent cluster to a distributed system. This immediately introduces the classical distributed-system challenges: task decomposition and scheduling, state synchronization and consistency, fault detection and recovery, deadlock avoidance, and load balancing.

5.6.3 Why the Analogy Holds

First, both face the “communication overhead versus parallelism benefit” trade-off. In distributed systems, Amdahl’s law tells us that parallel speedup is limited by the serial fraction. In multi-agent systems, Law III (the agent speedup law) predicts analogous behavior—when orchestration efficiency E is insufficient, simply adding more agents yields diminishing returns. AutoGen’s multi-agent experiments validate this prediction: increasing the agent count from 2 to 8 produced progressively smaller reductions in task completion time [?]. Industrial case studies further demonstrate that coordination topology significantly affects E : Fountain adopted a hub-and-spoke topology (a central Copilot coordinating sub-agents for screening, document generation, and sentiment analysis), achieving 50% faster candidate screening and full configuration within 72 hours [?]; MetaGPT employs a pipeline topology (SOP-driven sequential collaboration); AutoGen uses a peer-to-peer topology (conversational coordination). Different topologies yield different E values across task scenarios, implying that the E in Law III is not a fixed scalar but a design variable coupled to the coordination structure.

Second, both require coordination mechanisms. Distributed systems use consensus protocols such as Raft [?] to help multiple nodes agree on decisions; multi-agent systems need “debate,” “voting,” or “hierarchical arbitration” mechanisms for multiple agents to converge on a conclusion.

Third, both face a consistency–availability tension. The CAP theorem states that a distributed system cannot simultaneously guarantee consistency, availability, and partition tolerance [?]. In multi-agent systems, an analogous tension exists between “semantic consistency” (whether agents share the same understanding of a fact) and “response speed” (whether to wait for all agents to finish reasoning before producing a conclusion).

5.6.4 Essential Differences

First, communication bandwidth and precision differ fundamentally. Distributed nodes communicate through structured protocols (e.g., gRPC with Protocol Buffers), yielding precise, lossless message semantics. Agents communicate through natural language or semi-structured messages, where information may be misinterpreted, omitted, or distorted during transmission—*semantic noise* is a challenge unique to agent-based distributed systems.

Second, failure modes differ. Distributed-node failures are typically binary (online or

offline); agent “failures” can be *gradual*—the agent has not crashed, but its reasoning quality degrades, it drifts from the task objective, or it begins to hallucinate. Detecting “whether an agent is executing the task correctly” is far harder than detecting “whether a node is alive.”

Third, combinatorial emergence introduces uncertainty. The behavior of a distributed system is the deterministic composition of its nodes’ behaviors; multi-agent systems may produce **emergent effects**—each agent executes correctly in isolation, yet the collective outcome is unpredictable. This places new demands on system debuggability and accountability.

6 The Intelligent Computing Architecture Model (ICAM)

The analogy framework developed in the preceding four sections provides valuable intuition, but intuition alone cannot discipline engineering design. The enduring strength of classical computer architecture rests on its possession of **formal models**: from the hardware–software contract embodied in the instruction set architecture (ISA), to the locality theory governing memory hierarchies, to the quantitative analysis framework of Amdahl’s Law. These models give architects a shared, precise vocabulary for reasoning about trade-offs.

This section **formalizes** the analogies above into the *Intelligent Computing Architecture Model* (ICAM), comprising a six-layer functional hierarchy, inter-layer interface contracts, and six design axioms. Together, they provide the conceptual foundation for the quantitative laws developed in Section ??.

6.1 Six-Layer Hierarchy

The central organizing principle of classical computer architecture is **layered abstraction**: hardware is hidden behind the ISA, the operating system exposes services through system calls, and user applications need only invoke library functions. Each layer interacts solely with its immediate neighbors and remains oblivious to the implementation details of distant layers. This separation of concerns keeps system complexity tractable and enables independent evolution at every level.

ICAM transfers this principle to the model-native computing domain by defining six functional layers.

Definition 6.1 (Intelligent Computing Architecture Model — ICAM). *The Intelligent Computing Architecture Model (ICAM) organizes a model-native computing system into six functional layers, each interacting with adjacent layers through well-defined interface contracts:*

1. **L1 — Physical Execution Layer.** GPU, TPU, ASIC, and compute-in-memory hardware that execute matrix multiplications and attention computations. This layer is the analog of the ALU and floating-point unit in classical architecture: it carries out compute instructions without regard for their semantic meaning. The Transformer Engine in the NVIDIA H100, which accelerates FP8 matrix multiplication at the hardware level, is a representative L1 component[?].
2. **L2 — Inference Serving Layer.** Model weight loading, KV cache management, continuous batching, and prefill–decode scheduling. This layer parallels the microarchitecture and cache hierarchy in classical systems: it manages compute resources and hot data so that invocations from upper layers execute as efficiently as possible. vLLM’s PagedAttention, which enables on-demand KV cache allocation and sharing[?], and SGLang’s RadixAttention, which reuses cached prefixes across requests[?], are both representative L2 implementations.
3. **L3 — Context Management Layer.** Hot/warm/cold memory tiering, context compilation, summarization and retrieval, and semantic virtual memory. This layer is the analog

of the virtual memory subsystem in classical operating systems: it virtualizes the limited physical context window into a larger logical working space. MemGPT’s virtual context management treats the context window as main memory and external storage as disk, enabling the model to autonomously manage memory swap-in and swap-out[?]; MemoryOS borrows OS memory-management mechanisms to design a tiered memory architecture[?]. Both are representative L3 implementations.

4. **L4 — Semantic Interface Layer.** Tool schemas (the Tool ABI), Memory APIs, and Trace APIs. This layer parallels the system-call interface and ABI in classical architecture: it defines the uniform protocols through which an intelligent system accesses external capabilities. The Model Context Protocol (MCP) standardizes communication between models and tools[?], while the Agent-to-Agent (A2A) protocol defines interoperability between agents[?]. Both are representative L4 implementations.
5. **L5 — Orchestration Layer.** Task planning, agent scheduling, permission enforcement, failure recovery, and state commitment. This layer is the analog of the operating system kernel: it is responsible for scheduling tasks, allocating resources, enforcing isolation, and providing fault tolerance. OpenAI Codex submits agent tasks to sandboxes for execution and supports the creation and coordination of sub-agents[?, ?]; Claude Code extends agent capabilities through a skill mechanism and connects to external tools via MCP[?, ?]. Both are representative L5 implementations.
6. **L6 — Application Layer.** User tasks, agent applications, and workflow automation. This layer parallels the user-space application layer in classical systems: it carries end-user intent. SWE-agent autonomously resolves GitHub issues[?]; OpenHands serves as a general-purpose software-development agent[?]. It is worth noting that the L6 user base is rapidly expanding from professional engineers to domain experts: Anthropic’s legal team used Claude to reduce marketing-review turnaround from two to three days to under 24 hours; Zapier achieved 89% organization-wide AI adoption and deployed over 800 internal agents[?]. This trend underscores the need for richer interaction modes and stronger built-in guardrails at L6.

The ICAM layering is not an arbitrary decomposition. It obeys a single organizing principle: each layer provides services to the layer above and depends solely on the interface of the layer below—precisely the philosophy underlying network protocol stacks and operating-system hierarchies. The resulting benefit is **separation of concerns**: an L2 engineer optimizing KV cache policies need not understand agent orchestration logic, and an L5 developer designing task schedulers need not comprehend the hardware details of attention computation.

Table ?? maps each ICAM layer to its classical counterpart, summarizes its responsibilities, highlights the core difference, and lists representative implementations.

6.2 Inter-Layer Interface Contracts

The sustained evolution of classical computer architecture hinges on the **stability of inter-layer interfaces**. The ISA defines the contract between hardware and software: Intel’s processors evolved from the 8086 to today’s Core Ultra with sweeping microarchitectural changes, yet the x86 ISA maintained backward compatibility throughout. POSIX defines the contract between applications and the operating system: the Linux kernel progressed from 2.0 to 6.x, yet user applications required virtually no modification. The principle of “stable interfaces, mutable implementations” allows each layer to be optimized independently.

ICAM similarly defines an interface contract for each pair of adjacent layers. The following subsections specify the operations, invariants, and metrics for each interface.

6.2.1 L1–L2 Interface: Tensor Operation Contract

L2 requests tensor operations from L1. In plain terms, this interface resembles an application invoking a GPU driver: the upper layer says “compute this matrix multiplication,” and the lower layer executes it efficiently and returns the result.

- **Operations.** `forward(batch_tokens) → logits`. Input a batch of token embedding vectors; receive the corresponding logit probability distributions.
- **Invariants.** Compute precision must not fall below the configured minimum (e.g., FP16/BF16).
- **Metrics.** FLOPS utilization, memory-bandwidth utilization, TTFT (Time to First Token), and TPOT (Time Per Output Token).

6.2.2 L2–L3 Interface: KV Block Management Contract

L3 requests allocation, loading, and eviction of KV blocks from L2. This interface is analogous to an application calling `malloc/free`: the upper layer says “I need a block of memory to store KV cache,” and the lower layer allocates physical space, handles fragmentation, and reclaims capacity when it runs short.

- **Operations.** `alloc, load(prefix_hash), evict(policy)`. Allocate a new KV block, load an existing block by prefix hash, and reclaim inactive blocks according to a replacement policy.
- **Invariants.** KV blocks required by the current inference step must not be lost—analogous to the operating-system guarantee that a memory page actively referenced by a process will not be swapped out.
- **Metrics.** KV hit rate H (the core parameter in Law I, Section ??), peak GPU memory usage, and page-replacement count.

6.2.3 L3–L4 Interface: Context Loading Contract

L4 requests context loading from L3. This interface resembles an application memory-mapping a file via `mmap`: the upper layer says “I need information about X,” and the lower layer retrieves, compresses, and loads the relevant content from external storage into the context window.

- **Operations.** `read(query), prefetch(hint), compact, evict`. Retrieve context by query, prefetch based on a hint, compress context, and evict inactive context.
- **Invariants.** Returned content must carry provenance tracking and freshness annotations—analogous to a file system guaranteeing that reads return the most recent committed version.
- **Metrics.** Retrieval recall, context compression ratio, and loading latency.

6.2.4 L4–L5 Interface: Capability Invocation Contract

L5 requests tool invocations and memory reads/writes through L4. This interface is analogous to an application issuing system calls to the operating system: the upper layer says “I need to read/write a file” (corresponding to memory access) or “I need network access” (corresponding to tool invocation), and the lower layer enforces permission checks and audit logging.

- **Operations.** `call(tool, args, capability_token), query(memory)`. Invoke a tool with a capability token; query memory.
- **Invariants.** Tool invocations without a valid capability token are rejected (analogous to a process lacking file-open permissions); side-effecting operations are auditable[?].
- **Metrics.** Tool-call success rate and permission-denial rate.

6.2.5 L5–L6 Interface: Task Submission Contract

L6 submits tasks to L5. This interface resembles a user launching a program from the command line: the upper layer says “please complete this task,” and the lower layer handles queuing, execution, monitoring, and result delivery.

- **Operations.** `submit(task_spec)`, `poll`, `cancel`. Submit a task specification, poll for status, and cancel a running task.
- **Invariants.** Task execution produces a traceable audit trail; on failure, side effects are rolled back—analogueous to database transaction atomicity.
- **Metrics.** Task completion rate and human-takeover rate.

An empirical constraint worth noting. Anthropic’s internal research reveals that engineers use AI for approximately 60% of their work, yet can “fully delegate” only 0–20% of tasks[?]. This *collaboration paradox* implies that the L5–L6 task submission contract cannot be modeled as a binary decision (delegate or not); it must instead reflect a spectrum of *delegation confidence*. The more verifiable a task and the less it depends on organizational context, the higher the delegation confidence. This observation connects directly to the dual-plane architecture: the deterministic control plane provides the auditable traces that increase verifiability and, in turn, raise delegation confidence.

6.3 Dual-Plane Cross-Mapping with ICAM

Section ?? introduced the dual-plane architecture comprising a probabilistic execution plane (the non-deterministic reasoning of LLMs) and a deterministic control plane (auditable, programmatic control). A natural question arises: how do the two planes relate to the six ICAM layers?

The answer is that they are **orthogonal dimensions**. ICAM decomposes the system *vertically* (from hardware to application), while the dual-plane architecture partitions it *horizontally* (from probabilistic to deterministic). Their intersection forms a 6×2 matrix.

Table ?? presents this cross-mapping.

Two structural features emerge from this mapping.

The probabilistic plane is weighted toward the bottom. It primarily spans L1 (physical execution) and L2 (inference serving) and permeates into L3 (semantic retrieval and context compilation). This is unsurprising: the core capabilities of LLMs—language understanding and generation—are concentrated in these layers.

The deterministic plane is weighted toward the top. It primarily covers L5 (orchestration) and the interface-contract portions of L4 (permission enforcement, auditing, capability annotations), extending into the state-management portions of L3 (memory retention policies, version stamps). System-level requirements for safety, reliability, and auditability demand deterministic guarantees.

L4 occupies a distinctive role as a *transition layer*: tool schemas and protocol specifications belong to the deterministic plane (they are formalized interface definitions), while the semantic interpretation of tool results belongs to the probabilistic plane (the LLM must understand natural-language return values). L6 (application layer) is a consumer of both planes: end users simultaneously receive intelligent output from the probabilistic plane and an auditable trace from the deterministic plane.

This cross-mapping resolves an apparent contradiction in the literature. Researchers focused on inference optimization regard the LLM as a compute engine at L1–L2[?, ?]; those focused on agent systems see it as an intelligent kernel at L5–L6[?, ?]; those focused on context management treat it as a memory processor at L3[?, ?]. These perspectives are not mutually contradictory—they are observing different layers of the same ICAM stack through different planes.

6.4 Design Axioms

Eight decades of classical computer architecture have distilled a set of enduring **design principles**: the principle of locality guides cache design, layered abstraction guides interface specification, and least privilege guides security architecture. We now transplant these principles into the model-native computing domain and, informed by its distinctive characteristics, propose six design axioms. For each axiom we identify its classical source, its embodiment within ICAM, and its concrete design implication.

Axiom 6.1 Locality Axiom

Model-native computation exhibits both **temporal locality** (recently accessed context fragments are likely to be accessed again) and **semantic locality** (semantically similar queries access similar context).

Intuition

Just as running programs tend to revisit the same set of memory pages—motivating LRU cache replacement—LLMs in multi-turn conversations tend to re-reference the same context fragments. A user who first asks “how do Python decorators work?” will typically follow up with questions on the same topic. This is semantic locality in action.

Classical source

The theoretical foundations of cache design[?].

ICAM embodiment

KV cache reuse at L2 (identical prefixes share KV cache entries), prefix caching at L2 (multiple requests share the KV cache of a common system prompt), and semantic caching at L3 (semantically similar queries reuse previously generated results).

Design implication

Every ICAM layer should employ locality-aware caching and prefetching. Law I (Section ??) quantifies this axiom into a computable formula.

Axiom 6.2 Layered Abstraction Axiom

Complexity should be managed through layering: each layer depends only on the interface contract of the layer below, never on its implementation details.

Intuition

Just as a user application need not know whether the CPU is RISC or CISC, or whether the cache is 4-way or 8-way set-associative, an agent’s orchestration logic need not know which inference engine serves the underlying model.

Classical source

ISA–microarchitecture separation; kernel–user-space separation[?, ?].

ICAM embodiment

A model upgrade (a change in L2’s implementation) must not break agent logic at L5; a tool-schema revision (an interface adjustment at L4) must not corrupt memory data at L3.

Design implication

Interface contracts between adjacent layers must be defined with sufficient formality to enable independent evolution. Breaking changes at any layer should be detectable through contract-compatibility tests rather than through end-to-end regressions.

Axiom 6.3 Probabilistic Execution Axiom

The core execution in model-native computing—the model’s forward pass—is probabilistic: identical inputs do not guarantee identical outputs.

Intuition

This is the **most fundamental difference** between model-native and classical computing. In classical systems, 2+2 always yields 4; in an LLM, posing the same question twice may produce different answers—this is not a bug but a feature. This inherent non-determinism invalidates traditional testing methodologies: we cannot verify system correctness by re-playing an identical input sequence; instead, we must define *semantic equivalence* as the acceptance criterion.

Classical source

No direct classical counterpart. This axiom is unique to model-native computing.

ICAM embodiment

The entire ICAM hierarchy must accommodate probabilistic behavior. Interface contracts at L4–L5 cannot demand fully deterministic tool-call outcomes; they must tolerate semantic-level variation. Failure-recovery mechanisms at L5 cannot rely on exact input replay; they require semantic-level checkpoints.

Design implication

Every interface contract, testing procedure, and reliability mechanism in the ICAM stack must be designed for a world where the primary compute substrate is stochastic. Deterministic verification must be replaced (or supplemented) by statistical quality bounds and semantic acceptance tests.

Axiom 6.4 Virtualization Axiom

Limited physical resources should be virtualized to present a larger logical resource, while preserving isolation and protection.

Intuition

Just as a 32-bit operating system uses virtual memory to give each process the illusion of a 4 GB address space (even when physical RAM is only 1 GB), a model-native system must give each agent the illusion of an unbounded context window (even when the physical context window is only 128 K tokens).

Classical source

Virtual memory and process isolation[?].

ICAM embodiment

MemGPT’s virtual context management[?]; PagedAttention’s page-based KV management[?], which organizes KV cache into “pages,” allocates them on demand, and reclaims inactive

pages—a design identical in structure to the operating system’s virtual memory manager; agent sandboxing[?], which isolates an agent’s execution environment from the host system, much as a process’s address space is isolated from other processes.

Design implication

Every bounded resource at every ICAM layer should be wrapped in a virtualization layer that (a) presents an apparently unbounded interface to the consumer above, and (b) enforces isolation between concurrent consumers.

Axiom 6.5 Least Privilege Axiom

Every execution unit should be granted only the minimal set of permissions required to complete its assigned task.

Intuition

Just as a text editor does not need administrator privileges to edit a file, a code-generation agent should not have the authority to delete the entire repository.

Classical source

Operating-system security principles[?].

ICAM embodiment

CaMeL’s capability-security mechanism[?], which assigns fine-grained capability tokens to each agent operation; Codex’s OS-level sandbox[?], which restricts an agent’s file-system and network access permissions.

Design implication

The L4–L5 interface must enforce per-operation permission checks via capability tokens (Section ??), and every L5 orchestration layer must support configurable permission policies that can be tightened or relaxed without modifying agent logic.

Axiom 6.6 Observability Axiom

Every side-effecting operation must produce a traceable, auditable event record.

Intuition

Just as a database writes a Write-Ahead Log (WAL) for every write operation, every tool invocation and file modification made by an agent should be recorded so that the rationale for each action can be reconstructed after the fact.

Classical source

System logging and performance monitoring[?].

ICAM embodiment

The tracing mechanism in the OpenAI Agents SDK[?], which logs each agent decision and tool invocation; the trace-grading methodology in agent-security research[?], which evaluates the safety of an agent’s action trajectory.

Design implication

The deterministic control plane must capture a complete, tamper-evident log of all operations that pass through the L4–L5 interface. This log serves dual purposes: it enables post-hoc debugging and compliance auditing, and it provides the delegation-confidence signal (Section ??) that determines which tasks can be safely delegated at the L5–L6 boundary.

7 Quantitative Laws

The enduring power of classical computer architecture lies in its **computable principles**. Amdahl’s Law tells us that if the serial fraction of a program is $(1 - f)$, then even an infinite speedup of the parallel portion yields an overall speedup no greater than $1/(1 - f)$ —a theoretical ceiling for parallel computing [?]. The Roofline model tells us that the peak performance of each compute core is governed jointly by its arithmetic intensity and available memory bandwidth, enabling engineers to diagnose whether a kernel is compute-bound or memory-bound at a glance [?]. The value of these laws lies not in their mathematical sophistication but in the **unifying quantitative language** they provide, enabling data-driven engineering decisions.

In this section we propose three quantitative laws for model-native computing. Formally, Laws I and III are isomorphic to Amdahl’s Law; semantically, however, they capture performance constraints unique to model-native computing. We explicitly acknowledge that Laws I and III are not mathematically novel discoveries but rather the **deliberate transplantation** of Amdahl’s analytical framework into the model-native regime, reinterpreted with domain-specific parameters. The value of this transplantation lies not in mathematical innovation but in furnishing a previously qualitative design space with computable decision criteria, transforming “what should I optimize first?” from intuition into quantitative analysis.

7.1 Law I: The Law of Semantic Locality

Law 7.1 (Law of Semantic Locality). *In autoregressive inference, the speedup S is governed by the KV-cache hit rate H and the KV-reuse speedup factor α :*

$$S = \frac{1}{(1 - H) + H \cdot \alpha^{-1}} \quad (2)$$

7.1.1 Intuitive Parameter Semantics

Before proceeding to the derivation, we develop intuition for the two core parameters.

- H (KV-cache hit rate): **The fraction of KV tensors that can be reused directly rather than recomputed from scratch.** Consider a customer-service chatbot: if a user engages in five consecutive turns on the same topic, the KV tensors produced during the first four turns can be reused by the fifth, and H will be close to 1. Conversely, if every conversation starts on a brand-new topic, previous KV caches are of no use and H approaches 0. The domain of H is $[0, 1]$.
- α (KV-reuse speedup factor): **How many times faster a cache hit is compared to a miss.** On a hit, the system simply loads existing KV vectors from memory in $O(1)$ time; on a miss, it must compute the full attention in $O(n^2)$ time. The parameter α quantifies this gap. In typical deployments, α ranges from 10 to 100.

7.1.2 Derivation

We begin from first principles.

Step 1: Decompose execution time. Let T denote the total wall-clock time for a single inference request. This time decomposes into two disjoint components:

$$T = T_{\text{miss}} + T_{\text{hit}} \quad (3)$$

where T_{miss} is the time spent on KV-cache misses (requiring full recomputation) and T_{hit} is the time spent on KV-cache hits (direct reuse of existing KV tensors).

Step 2: Express each component in terms of the hit rate. Let T_{total} be the time required when *no* cache is available—i.e., every token must be computed from scratch. When a cache is present, a fraction $(1 - H)$ of the work incurs misses while a fraction H enjoys hits. Because a hit is α times faster than a miss, the hit component costs only $1/\alpha$ of its uncached value:

$$T = T_{\text{total}} \cdot (1 - H) + T_{\text{total}} \cdot H \cdot \frac{1}{\alpha} \quad (4)$$

Collecting terms:

$$T = T_{\text{total}} \cdot \left[(1 - H) + \frac{H}{\alpha} \right] \quad (5)$$

Step 3: Define the speedup. The speedup S is the ratio of uncached to cached execution time:

$$S = \frac{T_{\text{total}}}{T} = \frac{T_{\text{total}}}{T_{\text{total}} \cdot [(1 - H) + H/\alpha]} = \frac{1}{(1 - H) + H/\alpha} \quad (6)$$

which is exactly Equation (??).

7.1.3 Limit Behavior

- When $H \rightarrow 1$ (nearly universal hits): $S \rightarrow 1/(0 + 1/\alpha) = \alpha$. The speedup ceiling is governed entirely by reuse efficiency—even with 100% hits, the maximum speedup is α .
- When $H \rightarrow 0$ (nearly universal misses): $S \rightarrow 1/(1 + 0) = 1$. No speedup whatsoever, degenerating to the uncached baseline.
- When $\alpha \rightarrow \infty$ (hits become instantaneous): $S \rightarrow 1/(1 - H)$. The speedup ceiling is governed entirely by the hit rate—even if hits are “free,” misses still consume time.

7.1.4 Numerical Example

Consider an LLM inference service with $\alpha = 10$ (a hit is ten times faster than a miss). We examine how H affects S :

- $H = 0.5$ (half of all KV tensors are cached): $S = 1/(0.5 + 0.05) = 1.82\times$ speedup.
- $H = 0.8$ (80% hit rate): $S = 1/(0.2 + 0.08) = 3.57\times$ speedup.
- $H = 0.95$ (95% hit rate): $S = 1/(0.05 + 0.0095) = 16.8\times$ speedup.

Raising H from 0.5 to 0.8 yields roughly a $2\times$ improvement, whereas raising H from 0.8 to 0.95 yields roughly a $4.7\times$ improvement—**increasing marginal returns**. Once a reasonable cache foundation exists, every additional percentage point of hit rate pays disproportionate dividends.

7.1.5 Correspondence to Amdahl’s Law

Equation (??) is mathematically isomorphic to Amdahl’s Law:

$$S_{\text{Amdahl}} = \frac{1}{(1 - f) + f/p} \quad (7)$$

where f is the fraction of work that can be accelerated and p is the per-unit speedup. The correspondence is: $f \leftrightarrow H$ (the accelerable fraction equals the cache-hit fraction) and $p \leftrightarrow \alpha$ (the per-unit speedup equals the KV-reuse factor).

Design implication. There are two levers for improving inference throughput: increasing H (through better caching policies, prefix sharing, or semantic caching [?]) and increasing α (through faster KV loading, PagedAttention [?], or KV quantization [?]). These two levers are **complementary**: when H is already high, further increases in H yield large marginal gains;

when H is low, raising α alone is ineffective, and the priority should be improving the caching strategy first.

7.1.6 Empirical Validation

Validating Law I requires two pieces of data: the speedup reported by a system and a plausible value of α that allows us to back-solve for H .

1. **vLLM / PagedAttention.** Kwon et al. report that vLLM achieves 2–4 $\times$ throughput improvement over baseline inference systems [?]. Assuming $\alpha \approx 10$ (a conservative estimate: KV loading is roughly ten times faster than recomputing attention from scratch), solving Equation (??) yields $H \approx 0.56$ – 0.83 . This implies that vLLM’s PagedAttention mechanism enables 56–83% of KV tensors to be reused rather than recomputed.
2. **Prompt Cache.** Gim et al. report time-to-first-token (TTFT) reductions of 8 $\times$ to 60 $\times$ [?], corresponding to S values of 8 to 60. Taking $\alpha \approx 60$ (semantic caching allows the system to skip large swaths of attention computation), the implied H ranges from 0.75 to approximately 0.98. For highly structured prompts (e.g., a fixed system prompt followed by a user template), cache hit rates can be extremely high.

7.2 Law II: The Context Budget Law

Law 7.2 (Context Budget Law). *The effective working set W_{eff} of a language model is jointly constrained by the context window size C and the attention retention rate $\beta(L)$:*

$$W_{\text{eff}} \leq C \cdot \beta(L) \quad (8)$$

where C is the model’s nominal context window size (e.g., 128K, 1M tokens), L is the positional distance of information within the window, and $\beta(L) \in [0, 1]$ measures the probability that information at position L is effectively utilized by the model.

7.2.1 Intuitive Parameter Semantics

- C (context window size): The model’s “physical memory capacity.” For example, GPT-4-Turbo has $C = 128\text{K}$ and Gemini 1.5 Pro has $C = 1\text{M}$. C determines how many tokens the model can “see” at once.
- $\beta(L)$ (attention retention rate): The central innovation of Law II. It measures **how much of the information placed at position L within the context window the model actually uses**. $\beta(L) = 1$ means information at that position is perfectly utilized; $\beta(L) = 0$ means it is effectively ignored.
- W_{eff} (effective working set): The model’s “truly usable memory.” Even if $C = 1\text{M}$, if $\beta(L)$ is low at certain positions, W_{eff} may be far smaller than C —analogous to purchasing a 1 TB hard drive but reliably using only 100 GB.

7.2.2 Why $\beta(L)$ Decays—the “Lost in the Middle” Phenomenon

$\beta(L)$ is not a constant but a function that decays with positional distance L . This empirical fact originates from a widely observed phenomenon known as the “**Lost in the Middle**” effect.

Liu et al. systematically studied this phenomenon in their seminal work [?]: they placed critical information at various positions within long contexts and measured whether the model could retrieve it. The results revealed a pronounced **U-shaped curve**—information at the **beginning** of the context (analogous to a primacy effect) and at the **end** (analogous to a

recency effect) was retrieved with high accuracy, while information in the **middle** suffered a significant drop in retrieval accuracy.

Subsequent studies have further quantified this decay. The RULER benchmark shows that retrieval accuracy drops from >95% at 4K tokens to <70% at 128K tokens [?]. Long-Bench v2 confirms that the decay is more severe for tasks requiring deep reasoning [?]. The LOFT study demonstrates that even models supporting ultra-long contexts exhibit significant under-utilization of their stated capacity [?]. Recent work suggests that the effective context length exploited by LLMs may be only 10–20% of the stated window.

7.2.3 Why an Exponential Decay Model

We adopt an exponential decay form to model $\beta(L)$:

$$\beta(L) \approx \beta_0 \cdot e^{-\lambda L/C} \quad (9)$$

Three considerations motivate this choice. **First**, the exponential is the simplest monotone decay function, and its single parameter λ has an intuitive physical meaning: larger λ implies faster decay and a shorter effective context. **Second**, attention weights are the output of a softmax, which inherently has exponential form; modeling their decay with an exponential function is structurally natural. **Third**, experimental data from benchmarks such as RULER exhibit approximately linear behavior on a log scale [?], consistent with exponential decay.

We emphasize that the precise value of λ depends on the specific model and task type, and we do not attempt a parametric fit here. The value of Law II lies in providing a **conceptual framework**—it establishes that $W_{\text{eff}} < C$, and frames the magnitude of this gap and strategies for closing it as subjects for future research.

7.2.4 Correspondence to Classical Working-Set Theory

Law II has a deep structural correspondence with Denning’s working-set model [?].

In 1968, Peter Denning proposed that the set of memory pages accessed by a process within a time window τ constitutes its **working set** $W(\tau)$. Denning’s central insight was that if physical memory is smaller than $W(\tau)$, the system suffers **thrashing**—the CPU spends most of its time waiting for pages to be swapped in and out rather than performing useful work [?, ?].

Equation (??) is the semantic analogue of the working-set model: C corresponds to physical memory size and W_{eff} to the process’s true working set. The crucial difference is that in the classical model, the “usefulness” of a page is binary (a process either accesses it or does not); in Law II, $\beta(L)$ is continuous—the model’s utilization of information at different positions within the context forms a gradual spectrum.

7.2.5 Design Implications

Law II carries three direct design implications:

(1) Diminishing marginal returns from simply enlarging C . Extending the context window (e.g., from 128K to 1M) does increase C , but if $\beta(L)$ decays rapidly with L , then W_{eff} grows far more slowly than C . Merely chasing a larger nominal context window is not a fundamental solution to “insufficient memory.”

(2) Critical information should be placed at high- β positions. Because $\beta(L)$ is highest at the beginning and end of the context, prompt engineers should concentrate critical information at these locations and compress or offload auxiliary information to a RAG system.

(3) **This provides quantitative justification for hierarchical memory (L3).** When C is fixed, the only way to break through the W_{eff} ceiling is via semantic virtual memory (the responsibility of L3)—swapping inactive context out to external storage and swapping active context in to high- β positions. MemGPT’s virtual context management [?] and MemoryOS’s hierarchical memory [?] embody precisely this strategy.

7.2.6 Empirical Validation

Validating Law II requires measuring the gap between a model’s stated context window C and its effective context W_{eff} .

1. **RULER benchmark.** Hsieh et al. designed RULER to test effective context via needle-in-a-haystack retrieval tasks at varying context lengths [?]. Even for models that nominally support 128K contexts, accuracy on complex retrieval tasks drops from $>95\%$ at 4K to $<70\%$ at 128K. This implies $W_{\text{eff}} \approx 0.7 \times 128\text{K} \approx 90\text{K}$, far below the stated C .
2. **LongBench v2.** Bai et al. find that decay is substantially faster for deep reasoning tasks (e.g., mathematical proofs, multi-step logic) than for simple retrieval [?]. This confirms that λ is not a fixed property of the model but varies with task type.

7.3 Law III: The Agent Speedup Law

Law 7.3 (Agent Speedup Law). *When multiple agents execute in parallel, the speedup S_{agent} is governed by the parallelizable fraction F , the number of sub-agents N , and the orchestration efficiency E :*

$$S_{\text{agent}} = \frac{1}{(1 - F) + \frac{F}{N \cdot E}} \quad (10)$$

7.3.1 Intuitive Parameter Semantics

- F (parallelizable fraction): **The fraction of the task that can be distributed across multiple agents for concurrent execution.** For example, a “compare three papers” task allows each paper’s summary to be extracted by a separate agent simultaneously (F is high); a “compile, test, deploy in strict sequence” task forces most steps to execute serially (F is low). The domain of F is $[0, 1]$.
- N (number of sub-agents): The number of agents that can be dispatched concurrently, analogous to the number of processors in a distributed system. For instance, Codex supports creating multiple sub-agents for complex tasks [?], and Claude Code supports parallel sub-task processing through its sub-agent mechanism [?].
- E (orchestration efficiency): **The overhead factor incurred in coordinating multiple agents, $E \in (0, 1]$.** E captures the additional costs of multi-agent coordination: context synchronization (agents must share intermediate results), result merging (outputs from multiple agents must be integrated), conflict detection (multiple agents may modify the same resource), and permission negotiation (each agent’s permissions must be independently authorized). $E = 1$ indicates zero orchestration overhead; $E < 1$ indicates that overhead exists.

7.3.2 Derivation

The derivation parallels that of Law I but introduces the orchestration efficiency factor.

Step 1: Decompose execution time. Let T_{total} denote the execution time on a single agent. The task decomposes into a parallelizable fraction F and a serial fraction $(1 - F)$.

Step 2: Compute the time for each component.

- Serial component: $(1 - F) \cdot T_{\text{total}}$ (cannot be accelerated).
- Parallel component: distributed across N agents but subject to orchestration overhead E , so the effective speedup is $N \cdot E$ rather than N . The time is $F \cdot T_{\text{total}} / (N \cdot E)$.

Step 3: Aggregate and compute the speedup.

$$T = (1 - F) \cdot T_{\text{total}} + \frac{F \cdot T_{\text{total}}}{N \cdot E} \quad (11)$$

$$S_{\text{agent}} = \frac{T_{\text{total}}}{T} = \frac{1}{(1 - F) + \frac{F}{N \cdot E}} \quad (12)$$

Step 4: Verify degeneration. When $E = 1$ (no orchestration overhead), the formula degenerates to the standard Amdahl’s Law, confirming the derivation’s consistency.

7.3.3 Limit Behavior

- When $N \rightarrow \infty$: $S_{\text{agent}} \rightarrow 1/(1 - F)$. The speedup ceiling is determined entirely by the serial fraction—no matter how many agents are added, the serial portion cannot be parallelized.
- When $F \rightarrow 1$ (nearly everything is parallelizable): $S_{\text{agent}} \rightarrow N \cdot E$. Speedup is governed by the agent count and orchestration efficiency.
- When $E \rightarrow 0$ (extreme orchestration overhead): $S_{\text{agent}} \rightarrow 1/(1 - F + F/\infty) \rightarrow 1/(1 - F)$. Even with many agents, if coordination is extremely inefficient—agents spend most of their time negotiating rather than working—the outcome approaches that of each agent working independently and then waiting for the serial fraction to complete.

7.3.4 Numerical Example

Consider a code-review task with $F = 0.6$ (60% of the review work can be distributed) and $E = 0.7$ (30% overhead from coordination):

- $N = 2$: $S = 1/(0.4 + 0.6/(2 \times 0.7)) = 1/(0.4 + 0.43) = 1.20\times$.
- $N = 4$: $S = 1/(0.4 + 0.6/(4 \times 0.7)) = 1/(0.4 + 0.21) = 1.64\times$.
- $N = 8$: $S = 1/(0.4 + 0.6/(8 \times 0.7)) = 1/(0.4 + 0.11) = 1.96\times$.
- $N = 16$: $S = 1/(0.4 + 0.6/(16 \times 0.7)) = 1/(0.4 + 0.054) = 2.20\times$.

Increasing the agent count from 2 to 4 yields a 37% additional speedup, whereas increasing from 8 to 16 yields only 12%—**diminishing returns**, the hallmark of Amdahl-style scaling.

Industrial calibration with the Fountain system. Consider the Fountain Copilot system reported by Anthropic [?], in which a central orchestration agent coordinates three specialized sub-agents for candidate screening, document generation, and sentiment analysis ($N = 3$). If the baseline workflow requires 168 hours (one week) of manual effort for a full recruiting-center configuration and the agent-assisted pipeline completes it in 72 hours, then $S_{\text{agent}} = 168/72 = 2.33$. Substituting into Equation (??) with $N = 3$ yields a reasonable parameter estimate of $F \approx 0.7$ and $E \approx 0.65$. This industrial calibration is consistent in order of magnitude with the hypothetical $F = 0.6$, $E = 0.7$ values used above, demonstrating that the parameter space of Law III can be grounded with production data.

7.3.5 Comparison with Classical Amdahl’s Law

Law III differs from Amdahl’s Law in the introduction of the orchestration efficiency E :

$$S_{\text{Amdahl}} = \frac{1}{(1 - F) + F/N} \quad \text{vs.} \quad S_{\text{agent}} = \frac{1}{(1 - F) + F/(NE)} \quad (13)$$

The presence of E means the speedup of an agent system is **always strictly below** the classical Amdahl prediction (for $E < 1$). The reason is fundamental: in classical parallel computing, inter-processor communication overhead can often be mitigated through hardware (high-bandwidth interconnects) and algorithms (communication-avoiding designs). In agent systems, orchestration overhead encompasses not only communication costs but also **the inherent probabilistic nature of LLM inference**—agents coordinate through natural language, which is inherently less precise and more ambiguous than binary protocols.

7.3.6 Design Implications

Law III carries three direct design implications:

(1) Naively increasing the agent count does not guarantee proportional gains. When E is low (high coordination overhead), increasing N yields diminishing returns. This explains why simple “stack more agents” strategies often underperform expectations.

(2) Prioritize improving F and E . The return on investment is far higher for increasing F (better task decomposition, converting serial steps into parallelizable sub-tasks) and E (reducing orchestration overhead through standardized inter-agent communication protocols such as A2A [?]) than for simply increasing N .

(3) An optimal agent count exists. For given values of F and E , there exists a critical N^* beyond which the marginal speedup from adding another agent is less than the marginal cost (each additional agent consumes LLM inference resources). N^* can be estimated analytically by solving for the point at which $\partial S / \partial N$ falls below a cost-determined threshold.

7.3.7 Empirical Validation

Validating Law III requires real performance data from multi-agent systems.

1. **AutoGen.** Wu et al. report experimental data for multi-agent systems showing that as the agent count increases from 2 to 8, the rate of reduction in task completion time progressively diminishes [?]
—consistent with Law III’s prediction that S_{agent} exhibits diminishing growth as N increases.
2. **GTA-2 benchmark.** Wang et al. report that the open-workflow success rate of top-performing models on the GTA-2 benchmark is only 14.39% [?]. This suggests that current systems suffer from both low F (inadequate task-decomposition capability leaves most work serial) and low E (poor inter-agent coordination efficiency).
3. **Language Model Teams.** A recent study models multi-agent LLM teams as distributed systems and analyzes their speedup directly through an Amdahl’s Law framework [?]. The authors find that highly parallelizable tasks (e.g., independent document summarization) achieve near-linear speedup, whereas strongly dependent tasks (e.g., multi-step reasoning) exhibit a speedup approaching 1—fully consistent with the role of F in Law III.

7.4 A Unifying Perspective

Table ?? summarizes the core formula, supporting evidence, and open questions for each of the three laws.

Figure ?? plots the characteristic curves of the three quantitative laws.

The three laws share a common theme: each reveals a **performance ceiling** in model-native computing and points toward strategies for approaching it. Law I defines the theoretical limit of KV-cache optimization. Law II quantifies the actual returns from extending the context window. Law III delineates the effective boundary for scaling up the number of agents. To-

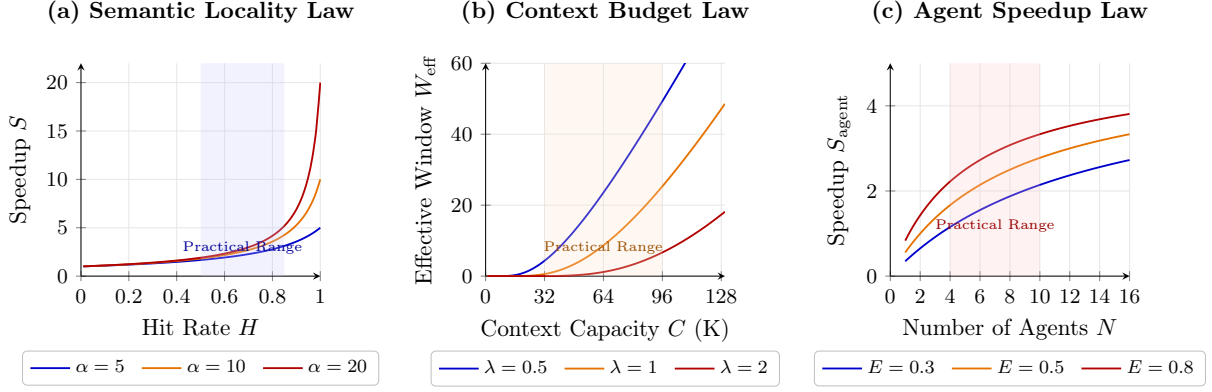


Figure 7: Characteristic curves of three quantitative laws. (a) Semantic Locality Law: $S = 1/((1 - H) + H/\alpha)$. (b) Context Budget Law: $W_{\text{eff}} = C \cdot e^{-\lambda C_0/C}$. (c) Agent Speedup Law: $S = 1/((1 - F) + F/(NE))$ with $F = 0.8$. Shaded areas indicate typical operating ranges.

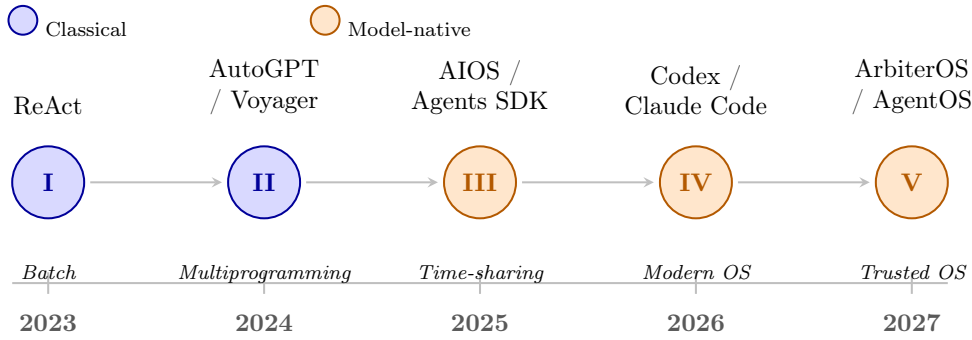


Figure 8: Five-generation evolution timeline of agent frameworks (2023–2026)

gether, these three laws form the **quantitative backbone** of the ICAM model, translating the qualitative design principles of Section ?? into computable engineering targets.

8 Agent Framework Evolution: From ReAct to the Model-Native OS

An agent framework is a software system built around a large language model that enables autonomous planning, tool invocation, state management, and complex task completion. Between 2022 and 2025, these frameworks have undergone a systematic evolution that mirrors the historical progression of operating systems—from single-task batch processing, through timesharing and multitasking, to protected and governed execution environments. This section traces five distinct generations of this evolution, examining each along three dimensions: its **core architectural contribution**, its **representative systems and their concrete mechanisms**, and its **structural correspondence to classical operating system history**.

Figure ?? presents the timeline of the five-generation agent framework evolution.

8.1 Generation I: Establishing the Execution Paradigm (2022–2023)

Core contribution. The first generation of agent frameworks established the interleaved *Thought–Action–Observation* loop as the fundamental execution paradigm. Prior to this, LLM usage followed a single-turn, question-answer pattern: the user supplied a prompt, and the model returned a complete response—analogueous to submitting a batch job and waiting for its output.

Generation I broke this limitation by placing the model inside an active loop in which it could **reason** (analyze the current state and decide on the next step), **act** (invoke a tool or execute an operation), and **observe** (read the tool’s return value), before entering the next iteration.

Representative systems. ReAct [?] is the foundational work of this generation. Its core mechanism injects a “Thought” field into the prompt template, requiring the model to emit an explicit reasoning trace before every tool invocation. When asked, for example, “What is Shakespeare’s longest play?”, the model does not guess the answer outright; instead, it first outputs “I need to search for the lengths of all Shakespeare’s plays,” calls a search tool, and then reasons over the results. This “think before you act” discipline, while seemingly simple, effectively mitigates the *hallucinated tool calls* that arise when models generate tool invocations without intermediate deliberation. Toolformer [?] approached the problem from a complementary angle: it taught the model **when and how** to invoke tools by injecting tool-call demonstrations into the training corpus, thereby learning tool usage as a self-supervised capability.

OS analogy. Generation I frameworks correspond to the **single-task batch-processing systems** of the 1950s, such as the Fortran Monitor System on the IBM 704. The system handles one task at a time, to completion, before starting the next; there is no concurrency, no interrupts, and no persistent state across tasks. The ReAct loop is the simplest possible “fetch–decode–execute” pipeline, with each iteration analogous to a single instruction cycle. There is no resource management whatsoever: no task scheduler (only one task exists), no memory management (the context window *is* the entire “memory”), and no I/O abstraction (tool calls are hard-coded).

8.2 Generation II: Persistent Multi-Step Agents (2023)

Core contribution. The defining advance of Generation II is **persistence** and **autonomous goal decomposition**. In Generation I, each user query triggers an independent reasoning–action loop, and all intermediate state vanishes upon completion. Generation II frameworks allow agents to run continuously, autonomously decompose high-level goals into sequences of subtasks, maintain intermediate state across steps, and pass information between subtasks—in effect, endowing the agent with a working memory.

Representative systems. AutoGPT [?] is the most widely recognized effort of this generation. Given a high-level objective such as “survey recent advances in quantum computing and write a report,” AutoGPT autonomously decomposes it into subtasks—searching papers, extracting key findings, organizing an outline, and drafting the report—and executes them in sequence. It introduces long-term memory backed by a vector database to store intermediate results across steps, enabling the agent to “remember” what it has already done. However, AutoGPT also exposed Generation II’s central weakness: the absence of failure recovery. Once a subtask goes wrong, the entire chain tends to spiral into infinite loops or accumulate compounding errors.

Voyager [?] made a critical improvement within the Minecraft environment by introducing a **skill library**—encapsulating successfully completed subtasks as reusable “skills” analogous to function definitions—and an **automatic curriculum** that generates the next learning objective based on the agent’s current capabilities, much like a syllabus. This allows the agent to accumulate experience incrementally rather than starting from scratch each time. BabyAGI implemented a cleaner task-queue manager: maintaining a prioritized to-do list, executing tasks in order, and dynamically spawning new tasks based on execution results.

OS analogy. Generation II corresponds to **multiprogramming** in the 1960s: the system begins to manage complex multi-step state, and tasks can share and pass information. AutoGPT’s task queue is a primitive “job scheduler”; Voyager’s skill library resembles an “executable program library”; BabyAGI’s priority ordering parallels early “job scheduling policies.” Compared with mature operating systems, however, Generation II lacks three critical mechanisms:

isolation (different tasks may interfere with each other), **preemptive scheduling** (a stalled task blocks the entire system), and **protection** (a failed task can corrupt the state of other tasks).

8.3 Generation III: General-Purpose Runtimes (2023–2024)

Core contribution. Generation III frameworks systematically introduce **scheduling**, **isolation**, and **resource management**, upgrading the agent framework from a single-script automation tool into a general-purpose runtime platform. The key indicator is that an agent is no longer merely a serial loop; it becomes a schedulable, suspendable, resumable “process.” Multiple agents can run concurrently without mutual interference, and the system explicitly manages computational resources—**inference budgets**, **context space**, and **tool-call rate limits**.

Representative systems. AIOS [?] constructed a complete agent operating system kernel organized into six subsystems: an Agent Scheduler for concurrent multi-agent scheduling, a Context Manager for context isolation and sharing across agents, a Memory Manager for hierarchical memory, a Tool Manager for tool registration and permission control, an I/O Manager for input–output mediation, and a Knowledge Manager for knowledge-base integration. Each agent is encapsulated as a “process” with its own state space and resource budget.

The OpenAI Agents SDK [?] pursued a more engineering-driven approach, packaging orchestration, state management, approval workflows, agent handoffs, and full-chain tracing into a code-first runtime. Developers define agent behavioral specifications, permission boundaries, and collaboration protocols in Python, rather than encoding them implicitly through prompt templates.

AutoGen [?] proposed a multi-agent conversational framework in which multiple agents collaborate on complex tasks through structured message passing, each assuming a distinct role (programmer, tester, project manager)—analogous to multi-process cooperation.

OS analogy. Generation III corresponds to the **time-sharing operating systems** of the 1960s–70s, such as Multics and Unix. The core breakthrough of time-sharing OSes was providing each user (process) with an independent address space and fair CPU time slices, enabling multiple users to safely share a single computer. AIOS’s Agent Scheduler corresponds to the process scheduler of a time-sharing OS; its Context Manager maps to the address isolation provided by the memory management unit (MMU); and the Tool Manager’s permission controls parallel Unix’s file permission system. The Agents SDK’s tracing mechanism mirrors OS-level system-call auditing via `strace` or `auditd`.

8.4 Generation IV: Engineering-Grade Operating Systems (2024–2025)

Core contribution. The hallmark of Generation IV is **deep integration with real-world software engineering environments**. Whereas the first three generations validated their concepts primarily in laboratory or constrained settings, Generation IV confronts real codebases, real terminal environments, real file systems, and real external services. This forces the system to address problems that earlier generations could sidestep: large repositories whose context far exceeds the model’s window, tool calls that may produce irreversible side effects, concurrent edits that can introduce conflicts, and real projects that demand permission controls and audit trails.

Representative systems. OpenAI Codex [?] is a cloud-based coding agent that runs inside an isolated sandbox: it checks out the repository in a dedicated container, performs reads, writes, and command execution, and submits a diff for human review upon completion. Codex introduced a sub-agent mechanism [?]: the primary agent can dispatch subtasks to specialized

sub-agents, each running in its own sandbox, with results merged after approval—analogous to Unix’s `fork()` creating child processes. Codex also supports skills packages [?] and project-level instruction files (`AGENTS.md`) [?], enabling reusable configurations across projects.

Claude Code [?] adopts a complementary design philosophy: it runs in the developer’s local terminal, connects to external tools via the Model Context Protocol (MCP) [?], defaults to read-only access (unless the user explicitly authorizes writes), and supports custom hooks that intercept and audit every tool call [?]. Claude Code’s memory system [?] distinguishes between project-level memory (`CLAUDE.md`, analogous to a system configuration file) and user-level memory (cross-project personal preferences, analogous to dotfiles in the user’s home directory).

OpenHands [?] provides an open agent platform supporting multiple model backends and sandbox environments; its OpenHands Index has become one of the standard benchmarks for evaluating coding agents. OSWorld [?] advanced this direction from an evaluation standpoint by constructing real operating system environments—complete with file managers, browsers, and terminals—and requiring agents to complete open-ended tasks within authentic GUI settings, thereby exposing agents’ limitations when confronted with real-world complexity. Devin [?] further exemplified Generation IV’s engineering orientation: as an “AI software engineer,” it integrates a code editor, browser, and terminal, and is capable of autonomous planning, coding, debugging, and deployment.

OS analogy. Generation IV corresponds to the **modern operating systems** of the 1980s–90s, such as Unix System V, early Linux, and Windows NT. The defining characteristic of modern OSes is their ability to handle real-world complexity: multitasking schedulers (e.g., the CFS scheduler [?]), virtual memory with demand paging, network protocol stacks, device driver frameworks, and security auditing. Codex’s sandbox maps to OS-level process isolation (each sub-agent runs in an independent container, akin to `chroot` or container technology); Claude Code’s hooks parallel Linux Security Modules (LSM) security hooks; Codex’s approval mechanism corresponds to Unix’s `sudo` privilege escalation; and OpenHands’s multi-model backend serves the role of a hardware abstraction layer (HAL).

8.5 Generation V: Governed Operating Systems (2025–Present)

Core contribution. The defining breakthrough of Generation V is the elevation of security and governance from post-hoc patches to **first-principles architectural design**. In the first four generations, security measures were additive: first make the agent functional, then bolt on sandboxes, approval gates, and audit logs. Generation V inverts this ordering: define security boundaries and governance rules first, then empower the agent to act within those boundaries. This paradigm shift parallels the historical transition of operating systems from “functional” to “trustworthy”—just as Multics introduced ring protection and mandatory access control in the 1960s, but it was not until the widespread adoption of SELinux and capability-based security that protection became an organic component of OS architecture [?].

Representative systems. ArbiterOS [?] explicitly articulates a layered architecture of “probabilistic CPU + deterministic governor/kernel”: the underlying LLM serves as the probabilistic processor that determines *what the agent can do*, while the governor above acts as the deterministic controller that decides *what the agent should do*. The governor constrains the agent’s behavioral boundaries through policy specifications, analogous to how an OS kernel defines a process’s capability envelope via system-call tables and permission bits.

AgentOS [?] introduces a reasoning kernel governed by structured operating system logic, building system-level intelligent behavior upward from token-level context management. Its central thesis is that an agent’s reasoning process should not be an uncontrolled “black-box loop,” but rather managed through structured abstractions and interfaces, much like an OS kernel.

UFO² [?], targeting desktop environments, organizes the Desktop AgentOS into three tiers: a HostAgent as the top-level coordinator (analogous to the `init` process), AppAgents—one per application—(analogous to service processes), and a GUI-API action layer at the bottom (analogous to the driver layer). This stratification enables independent management and auditing of agent operations across different applications.

Aura [?] realizes a security-first agent OS architecture on mobile devices, comprising a System Agent (with highest privilege), Sandboxed App Agents (one per application, akin to Android’s application sandbox), and an Agent Kernel responsible for permission arbitration and resource allocation. Aura emphasizes the unique risks of mobile environments: semantic input pollution (hijacking agent behavior through malicious inputs), memory taint propagation (compromised memory affecting all subsequent operations), and cross-application permission leakage.

CaMeL [?] derives its security model from first principles, proposing a capability-based model to defend against prompt-injection attacks. Its core insight draws directly from capability-based security in operating systems: an agent cannot directly execute any operation with side effects, but must obtain a temporary capability token from a deterministic capability manager. Each token authorizes only a single, specific operation (e.g., “read-only access to file X”) rather than granting blanket permissions. This mirrors the evolution in Unix from the coarse-grained root/non-root privilege model to the fine-grained POSIX capabilities and SELinux mandatory access controls.

IronClaw [?] systematically argues why AI agents require operating-system-grade protection. It observes that traditional software security assumes deterministic code with predictable behavior, whereas agent behavior is driven by probabilistic models. The attack surface thus expands from “code vulnerabilities” to “semantic vulnerabilities”: an adversary can hijack an agent’s behavior through natural-language prompts without exploiting any code defect.

The OWASP **Agentic Applications Top 10** [?], published in late 2025, further systematizes these threats into ten risk categories—including Agent Goal Hijack, Tool Misuse & Exploitation, and Identity & Privilege Abuse—providing a taxonomic foundation for the security design of Generation V frameworks.

OS analogy. Generation V corresponds to the inflection point at which operating systems transitioned from “functional” to “trustworthy.” In classical OS history, Unix V6 (1975) was a functional but insecure system: it lacked memory protection (any process could read or write any memory address) and privilege separation (the root user held unrestricted power). As networking and multi-user environments developed, security evolved from an “optional feature” to a “foundational requirement,” giving rise to Multics ring protection, SELinux mandatory access control, and formal verification of modern microkernels such as seL4 [?]. Generation V agent frameworks are undergoing the same transition: security is no longer merely “add a sandbox,” but requires a top-to-bottom redesign of the agent’s permission model, audit mechanisms, and failure-recovery strategies at the architectural level.

8.6 The Five Generations Through the Lens of Law III

Viewed through the agent speedup law (Law III) introduced in Section ??, the five generations of evolution correspond to a progressive increase in F (the parallelizable fraction) and E (the orchestration efficiency).

Recall the formal statement of Law III:

$$S_{\text{agent}} = \frac{1}{(1 - F) + \frac{F}{N \cdot E}} \quad (14)$$

where $F \in [0, 1]$ denotes the fraction of the task that can be decomposed for parallel execution, N is the number of schedulable agents, and $E \in (0, 1]$ is the orchestration efficiency ($E = 1$ means zero overhead; $E \rightarrow 0$ means overhead consumes all parallel benefit).

Quantitative trajectory across generations.

Generation I (ReAct). $F \approx 0$ (no task decomposition; single-agent serial execution), $N = 1$, $E \approx 1$ (no orchestration overhead), yielding $S_{\text{agent}} \approx 1$. This corresponds to the “uniprocessor, no parallelism” regime.

Generation II (AutoGPT/Voyager). $F \approx 0.1\text{--}0.2$ (a small fraction of subtasks can be separated), $N = 1$ (still a single agent), $E \approx 0.9$, yielding $S_{\text{agent}} \approx 1$. Decomposition capability begins to emerge, but with only one agent there is no opportunity for parallel speedup.

Generation III (AIOS/Agents SDK). $F \approx 0.3$ (moderate decomposition capability), $N \approx 2\text{--}4$ (multi-agent support emerges), $E \approx 0.7$ (scheduling and context switching introduce significant overhead), yielding $S_{\text{agent}} \approx 1/(0.7 + 0.3/2.8) \approx 1.27$. Multi-agent execution begins to deliver speedup, but orchestration overhead partially cancels the gains.

Generation IV (Codex/Claude Code). $F \approx 0.5$ (real-world software engineering tasks exhibit substantial decomposability), $N \approx 4\text{--}8$ (sub-agent concurrency), $E \approx 0.5$ (sandbox isolation, approval workflows, and state synchronization impose significant overhead), yielding $S_{\text{agent}} \approx 1/(0.5 + 0.5/4) \approx 1.6$. Speedup becomes substantial, yet orchestration overhead remains the binding constraint.

The execution time horizon of Generation IV systems is also expanding from minutes to hours. In a striking demonstration, engineers at Rakuten tested Claude Code on the vLLM project (12.5 million lines of code), where it ran autonomously for seven hours to complete an activation-vector extraction task, achieving 99.9% numerical accuracy [?]. This kind of long-duration autonomous execution introduces new architectural challenges—cross-hour KV-cache persistence, checkpoint/recovery protocols, and sustained resource management—that further reinforce the analogy between agent runtimes and operating systems.

The target: a true model-native OS. Achieving a genuine model-native operating system requires $F > 0.8$ and $E > 0.8$, which would yield $S_{\text{agent}} > 1/(0.2 + 0.8/8) \approx 3.3$. This target demands both high decomposability (high F) and low orchestration overhead (high E)—a dual requirement that no current system meets.

Recent benchmark results on GAIA [?], τ -bench [?], and SWE-bench [?] remind us how far current systems remain from this goal. On SWE-bench Verified, the best-performing agent systems as of late 2025 achieve a pass rate of approximately 70–75%, meaning that in roughly one quarter of real software engineering tasks, agents still cannot correctly decompose and execute the required work. On τ -bench, agent performance during multi-turn interactions with real users is even less stable, because the ambiguity of user instructions and the dynamism of the environment further depress both F and E .

9 Design Challenges: Performance, State, Consistency, and Security

Each generational leap in agent frameworks—from first-generation tool chains to fifth-generation autonomous agents—has exposed new systemic problems that neither the model itself nor ad hoc engineering workarounds can fully resolve. This section examines five cross-cutting design challenges through a uniform lens: (1) we describe a concrete scenario that illustrates the problem; (2) we analyze the root cause; (3) we survey current solutions; and (4) we draw an explicit analogy to a well-studied problem in classical computer architecture. Throughout, we trace each challenge back to the ICAM layers (Section ??) and the quantitative laws (Section ??), argu-

ing that the dual-plane architecture introduced in Section ?? is not merely an organizational convenience but a *necessary* structural response to these challenges.

9.1 The Latency–Throughput–Cost Trilemma

9.1.1 Scenario

Consider a code agent assisting a developer in fixing a bug that spans ten files. The agent must read multiple files (each read triggers a model inference call), understand the code structure, locate the fault, modify the code, run tests, inspect the results, and iterate. The entire workflow may involve 20–50 model calls, and the per-call latency directly determines the developer’s wait time. If a single call costs 2 seconds, the end-to-end latency can easily exceed one minute. If the serving provider provisions additional GPUs to reduce per-call latency, the cost per invocation rises proportionally. Conversely, if the provider increases batch sizes to amortize cost and improve throughput, individual requests queue longer, increasing latency. This is the *latency–throughput–cost trilemma*: reducing latency demands more resources (higher cost), improving throughput demands larger batches (higher latency), and controlling cost demands fewer resources (higher latency and lower throughput).

9.1.2 Root Cause

The root cause lies in a structural asymmetry inherent to autoregressive LLM serving. The **prefill** phase is *compute-bound*: the entire input sequence must be processed in one forward pass, keeping GPU compute units fully occupied, with FLOPS as the bottleneck. The **decode** phase is *memory-bandwidth-bound*: generating each new token requires reading the full model weights and the current KV cache—effectively streaming all parameters from memory on every clock cycle [?]. Gholami et al. quantify this imbalance: peak FLOPS grow at $3.0\times$ every two years, while DRAM bandwidth improves at only $1.6\times$ over the same period [?].

Because the two phases demand fundamentally different hardware resources, co-scheduling them in the same batch inevitably leaves some resource underutilized: prefill monopolizes compute while decode stalls on memory bandwidth. Agent workloads exacerbate this problem. Multiple sub-agents issue concurrent requests whose context lengths vary wildly—one may read a single function while another must ingest an entire repository—rendering static batching strategies inefficient.

9.1.3 Current Solutions

The research community has responded at multiple levels.

Scheduling-level optimizations. ORCA [?] introduces *iteration-level scheduling*: rather than waiting for an entire sequence to complete before admitting new requests, the scheduler checks after every decode step whether a new request can join the running batch, significantly improving GPU utilization. DistServe [?] goes further by *disaggregating* prefill and decode onto separate GPU groups, allowing each group to be independently optimized—high-FLOPS GPUs for prefill, high-bandwidth GPUs for decode. Sarathi-Serve [?] introduces stall-free scheduling via *chunked prefill*, which slices long inputs into fixed-size chunks and interleaves them with decode tasks, eliminating stalls caused by waiting for large prefills to complete.

Memory management optimizations. vLLM [?, ?] employs PagedAttention to organize the KV cache into fixed-size pages mapped through a block table to non-contiguous physical

memory, enabling on-demand allocation and sharing of KV cache. SGLang [?, ?] uses Radix-Attention to organize request prefixes in a radix tree, automatically identifying and reusing shared-prefix KV cache entries. TGI [?] and TensorRT-LLM [?] provide production-grade implementations of continuous batching and paged KV management. vAttention [?] offers an alternative approach: leveraging OS-level contiguous virtual memory with dynamic growth to support standard attention kernels directly, bypassing the need for specialized PagedAttention kernels.

9.1.4 Analogy to Classical Architecture

This challenge is a direct analog of the classical *memory wall*. Since the 1980s, CPU speeds have improved at roughly 50% per year while DRAM latency has improved at only 7% per year [?]. The architectural response—multi-level caches (L1/L2/L3), hardware prefetching, and write buffers—maps directly onto LLM serving: KV cache tiering corresponds to cache hierarchy, prefix caching to instruction cache sharing, and asynchronous decoding to write buffering. The familiar response-time-throughput trade-off in time-sharing operating systems is also analogous: interactive tasks demand low response time (low latency) while batch tasks demand high throughput (large batches), and the scheduler must balance both [?].

Within ICAM, this trilemma primarily spans L1 (physical execution) and L2 (inference serving). Law I (Section ??) quantifies the cache-side of this trade-off: the KV cache hit rate H directly determines the achievable speedup, and any scheduling or memory management optimization can be understood as an attempt to maximize H under the trilemma’s constraints.

9.2 State Management and Cross-Session Consistency

9.2.1 Scenario

Suppose a user works with the same agent over three consecutive days to build a project. On Day 1, the agent learns that the user prefers TypeScript over JavaScript. On Day 2, the user asks the agent to refactor the codebase; the agent must respect the preference learned on Day 1 while correctly understanding the current state of the code. On Day 3, the user discovers a new bug, and the agent must recall modifications made on both previous days—the bug may have been introduced by those very changes—while reasoning about the latest project state. To complicate matters further, suppose the user runs two agent instances concurrently on Day 2: one performing the refactor and another writing tests. Both agents may modify the same file, creating a conflict.

9.2.2 Root Cause

A traditional CPU is a *stateless* executor: given the same inputs and register state, the output is deterministic and reproducible. Intelligent systems, by contrast, must *persist* state across sessions—user preferences (“I prefer TypeScript”), task history (“yesterday we refactored the authentication module”), and external-world changes (“three new files were added to the repository”). Once persistent state is introduced, at least three categories of problems inevitably arise:

1. **Concurrent access to shared memory.** When multiple agents read and write shared memory simultaneously, how do we guarantee consistency? This is analogous to race conditions in multi-threaded programming.
2. **Delayed commits and conflict resolution.** Agent A modifies a file but has not yet committed the change; Agent B begins modifying the same file based on the old version.

Detecting and resolving such conflicts mirrors optimistic concurrency control in distributed systems.

3. **Memory staleness.** The fact “the project uses React” learned three days ago may be obsolete—the project has since migrated to Vue. Determining whether a memory is still valid is analogous to cache invalidation.

9.2.3 Current Solutions

The LongMemEval benchmark [?] demonstrates that long-term memory is not simply “storing more chat logs” but requires four distinct capabilities: information extraction (distilling key facts from conversation), temporal reasoning (understanding event ordering), knowledge updating (revising old memories when new information conflicts), and refusal (declining to answer when memory is insufficient).

MemGPT [?] borrows the paging and swap-in/swap-out mechanisms of OS virtual memory to manage context, dividing it into “main memory” (the active context window) and “external storage” (long-term memory in a vector database), with explicit edit and retrieve operations to move information between the two. MemoryOS [?] abstracts memory management as an OS memory management module, implementing structured memory encoding, retrieval, and update. A-MEM [?] proposes agentic memory management: memories are not passively stored strings but “active entities” whose creation, update, and deletion are governed by the agent itself. Letta [?] (formerly the MemGPT project) further elevates “stateful agents” to a core abstraction, explicitly separating conversational state, core memory, and archival memory.

From a distributed systems perspective, at least three state types must be distinguished [?, ?, ?]:

1. **Ephemeral state:** intermediate results of the current inference step, analogous to CPU registers; no persistence required.
2. **Session state with causal ordering:** message passing and state transfer among agents within the same task, requiring causal consistency guarantees analogous to those in distributed systems [?].
3. **Committed state with full auditability and replay:** permanent modifications to files, databases, or external systems, analogous to a database transaction’s commit point, supporting audit and rollback [?].

9.2.4 Temporal Decay in Long-Running Agents

Critically, the time horizon of agent execution is expanding from minutes to hours and even days. Recent industrial practice shows that agents can autonomously run for over seven hours to complete complex tasks [?]. Such cross-hour and cross-day execution introduces additional temporal decay challenges: the attention decay function $\beta(L)$ is no longer solely a function of context position but also depends on session duration and cross-step state accumulation. This means the three-way state classification above acquires a time dimension: cross-hour execution requires session-persistent KV caches (L2), context window management must support working-set restoration across checkpoints (L3), and the governance layer must define periodic checkpointing and human-recovery protocols (L5). Law II (Section ??), as currently formulated, does not capture this temporal dimension—an important direction for future work.

9.2.5 Analogy to Classical Architecture

These challenges map directly onto *virtual memory and distributed consistency* in classical systems. OS virtual memory uses page tables to map virtual to physical addresses and provides

swap mechanisms to create the illusion of unbounded memory—analogous to context management in agent systems. Distributed file systems such as GFS and Spanner [?] use Paxos/Raft [?] consensus protocols to guarantee multi-replica consistency—analogous to consistency in multi-agent shared memory. The CAP theorem [?] states that consistency, availability, and partition tolerance cannot be simultaneously achieved; agent memory management faces the same trade-off: strong consistency (synchronizing all replicas before acknowledging each operation) versus eventual consistency (tolerating brief inconsistency in exchange for lower latency).

Within ICAM, this challenge primarily spans L3 (context management) and L5 (orchestration). The dual-plane architecture naturally separates the problem: the probabilistic execution plane manages the working context (what to remember and what to forget), while the deterministic control plane enforces consistency guarantees and conflict-resolution protocols.

9.3 Interface Drift and Version Compatibility

9.3.1 Scenario

An agent is configured to manage a code repository via the GitHub API. One day, GitHub updates the API from v3 to v4: the endpoint `repos/:owner/:repo/pulls` returns a new response format, adds pagination parameters, and removes several fields. The agent’s tool calls begin returning errors or failing to parse responses. A more subtle scenario: the agent’s prompt template relies on the model producing output in a specific format (e.g., “always respond in JSON”), but a model upgrade subtly changes the output style, causing the downstream parser to crash repeatedly. Similar problems arise when MCP servers update their capability descriptions, when skill packs change their interfaces, or when memory file formats undergo version upgrades.

9.3.2 Root Cause

Classical computer systems achieve scalability in large part because their core interfaces—ISA, ABI, system calls, and file formats—remain *stable over decades*. The x86 instruction set has maintained backward compatibility from the 8086 (1978) through the latest processors in 2026; the POSIX standard, established in 1988, has preserved the semantics of `read()/write()/fork()` with almost no change [?]. This stability allows upper-layer software to be built with confidence that the ground beneath it will not shift unexpectedly.

The current agent ecosystem, by contrast, suffers from severe *interface drift*. At least four sources of drift can be identified:

1. **Model version drift:** a model upgrade may change the output format for the same prompt.
2. **Tool schema drift:** external APIs may update their interface descriptions and response formats without warning [?].
3. **Protocol drift:** protocols such as MCP and A2A are themselves under rapid iteration [?, ?].
4. **Skill pack drift:** reusable agent skills may behave differently across projects or model versions.

9.3.3 Current Solutions

OpenAI Codex provides project-level stability anchors through AGENTS.md [?] and the Skills mechanism [?]: each skill pack carries explicit version numbers and compatibility declarations. Claude Code achieves similar functionality through CLAUDE.md [?]. Interoperability protocols—MCP [?] for model-tool communication and A2A [?] for agent-agent interaction—

are attempting to establish standardized interfaces. The A2A protocol, built on JSON-RPC 2.0 over HTTP, defines standard workflows for agent discovery, capability negotiation, and task delegation [?]. However, these protocols are themselves evolving rapidly and have not yet achieved the stability of an ISA or POSIX.

9.3.4 Analogy to Classical Architecture

This challenge corresponds to *ABI stability and version management* in classical systems. Intel has maintained x86 backward compatibility for over 45 years; the Linux kernel has preserved system-call backward compatibility for over 30 years; even shared-library versioning has mature solutions (semantic versioning, symbol versioning). Model-native computing architectures must develop their own “version control discipline”: tool schemas need version numbers, model output formats need compatibility guarantees, skill packs need dependency management (analogous to npm’s `package.json`), and memory files need format migration tools (analogous to database schema migration).

Within ICAM, interface drift primarily affects L4 (semantic interface layer) and the L4–L5 boundary. The deterministic control plane must maintain version-aware interface contracts that insulate the probabilistic execution plane from upstream changes—precisely the role that ABI stability plays in shielding application software from hardware evolution.

9.4 Security, Privacy, and Least Privilege

9.4.1 Scenario

A code agent is helping a developer process user data. The agent must read database schemas that contain personally identifiable information (to understand the data model), modify code that handles user data, and run tests whose fixtures may include real user records. During this process, the agent may: (1) write sensitive data to log files or transmit it to a remote API (data exfiltration); (2) be manipulated by a malicious code comment or file content into performing unintended actions via prompt injection (goal hijacking); (3) use real user data in tests instead of synthetic fixtures (privacy violation). Worse still: since the agent can execute arbitrary shell commands, a carefully crafted prompt injection could cause it to run `rm -rf /` or upload sensitive files to an external server.

9.4.2 Root Cause

Once an agent can read and write files, execute commands, browse the web, and query databases, it is no longer merely a language model but an *agent* in the security sense—a principal with external action capability. This introduces attack surfaces that classical software security does not cover:

1. **Prompt injection.** An attacker embeds malicious instructions in tool-returned data, file contents, or web pages to hijack the agent’s behavior. Unlike classical code injection (e.g., SQL injection), prompt injection exploits not a logic flaw but the model’s inability to distinguish “instructions from the user” from “instructions embedded in data” [?, ?].
2. **Privilege over-provisioning.** Most current agents, when granted tool access, receive the tool’s *full* permission set (e.g., read/write access to the entire filesystem) rather than the minimum necessary privilege (e.g., read/write access only to specific project files).
3. **Memory taint propagation.** A successful prompt injection can write malicious information into the agent’s long-term memory; this “tainted” memory then influences the agent’s behavior across all subsequent sessions, creating a persistent backdoor [?].

9.4.3 Current Solutions

Industry and academia are constructing defenses along multiple dimensions.

Sandboxing. OpenAI Codex runs tasks by default inside an OS-enforced sandbox, where each task executes in an isolated container without access to the host filesystem or network [?]. Claude Code operates in read-only mode by default, requiring explicit user authorization for any write operation [?]. Aura establishes independent sandboxes for each app agent on mobile devices and restricts the agent kernel’s access to system resources [?].

Capability-based security models. CaMeL [?] applies the OS capability-security principle to defend against prompt injection: the agent cannot directly execute side-effecting operations but must obtain a “temporary pass” from a deterministic capability manager. Each pass authorizes exactly one specific operation (e.g., “read-only access to `src/main.py`”) and is consumed upon use. This fundamentally eliminates the possibility of privilege escalation via prompt injection, because permissions are governed by deterministic code rather than a probabilistic model.

Approval and audit. Codex’s approval policies [?] allow developers to specify which operations require human confirmation (e.g., “any `DELETE` operation must be approved”). The Agents SDK’s tracing mechanism [?] records the full context of every tool call, supporting post-hoc audit and replay.

Formal security. IronClaw [?] proposes a formal framework for agent security that explicitly models the agent’s behavior space and permission space, detecting out-of-bounds actions through static analysis and runtime monitoring. Christodorescu et al. [?] argue that agent security must draw on the full experience of operating system security: least privilege, isolation, auditing, and fail-safe defaults.

9.4.4 Analogy to Classical Architecture

These challenges correspond to *process security and permission management* in classical systems. Unix security rests on three core concepts: (1) user/kernel mode separation—user programs cannot access hardware directly and must go through system calls [?]; (2) file permissions (rwx)—each file has explicit read, write, and execute permissions; (3) root/non-root separation—privileged operations require explicit elevation. Agent security requires an analogous three-layer model: (1) probabilistic-plane/deterministic-plane separation—agents cannot directly execute side-effecting operations but must pass through the deterministic control plane’s approval channel; (2) tool capability annotations—each tool carries explicit capability declarations and permission requirements; (3) governance-layer privilege elevation—high-risk operations require explicit authorization or human confirmation.

9.4.5 Dual-Use Security Concerns

It is important to recognize that agent capabilities are inherently dual-use. The same capabilities that enable automated security auditing (defensive use) can also enable automated vulnerability discovery and exploitation (offensive use) [?]. This means the deterministic control plane of a dual-plane architecture must not only constrain its own agents’ behavior but also anticipate that adversaries possess equivalent dual-plane systems. Anthropic’s 2026 agentic security report predicts that “agentic cyber defense systems” will operate at machine speed [?], implying that

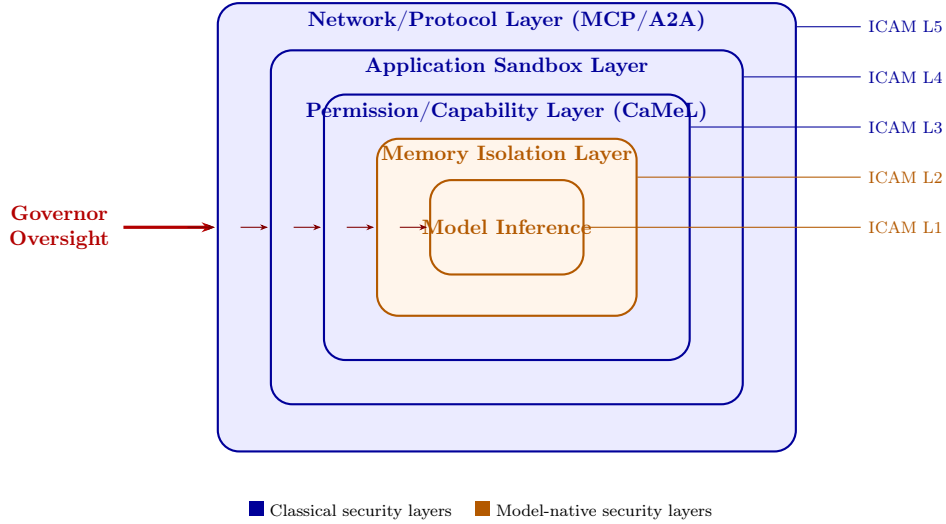


Figure 9: ICAM-layered security defense architecture

the L5 governance layer’s primitives must incorporate adversarial scenarios into their threat models from the outset.

Within ICAM, security spans all six layers but concentrates at L4 (tool capability annotations and call audits) and L5 (permission approval and failure recovery). The dual-plane architecture is essential here: the probabilistic plane handles the reasoning needed to interpret user intent, while the deterministic plane enforces the non-negotiable security invariants.

9.5 Governance as a First-Class Architectural Concern

9.5.1 The Core Problem

ArbiterOS [?] identifies the fundamental crisis in agent engineering as a structural contradiction: developers instinctively apply *deterministic software thinking* to drive a *probabilistic processor*. In traditional software engineering, code behavior is deterministic: identical inputs produce identical outputs, tests can cover all branches, and bugs can be caught with assertions. Agent behavior, however, is driven by a probabilistic model: the same prompt may yield different outputs across invocations, tests cannot cover all possible reasoning paths, and “bugs” manifest as hallucinations or instruction-following failures that resist detection by traditional assertions.

Aura [?] further demonstrates that once agents enter the OS and mobile environments, permission management (does the agent have access to the contacts list?), authentication (is the caller the genuine user or an attacker?), semantic input pollution (can a malicious SMS hijack the agent?), and memory taint propagation (how is tainted memory cleaned?) all become *kernel-level* concerns—no longer merely application-layer vulnerabilities but foundational security issues for the entire system architecture.

Figure ?? illustrates the ICAM-layered security protection architecture.

9.5.2 Governance Is Not a Retrofit

These observations imply that governance must not be treated as an after-the-fact security patch or a compliance checklist. Just as operating system designers learned—through decades of painful experience—that bolting on security after the fact is fundamentally unworkable [?], agent system governance must be embedded into the architecture from day one.

9.5.3 Per-Layer Governance in ICAM

Specifically, each layer of ICAM (Section ??) requires its own governance mechanisms:

- **L1—Physical Execution:** hardware-level inference precision guarantees and energy budgets. Analogous to CPU thermal monitoring and frequency throttling, L1 must ensure that inference precision does not fall below a safety threshold (e.g., quantization must not cause critical judgments to fail) and that GPU/TPU energy consumption remains within bounds.
- **L2—Inference Serving:** inference budgets and precision governance. Each agent’s inference calls should have budget caps (e.g., “at most 10 inference calls per sub-task”); exceeding the budget should trigger failure recovery rather than infinite retries. KV cache management must ensure that critical state is not inadvertently evicted.
- **L3—Context Management:** memory retention and deletion policies. Sensitive user information (passwords, credit card numbers) should be automatically detected and promptly removed, analogous to the GDPR’s “right to be forgotten” [?]. Memory entries should carry explicit time-to-live (TTL) annotations; expired memories should be automatically degraded or deleted.
- **L4—Semantic Interface:** tool capability annotations and call audits. Every external tool should carry a standardized capability declaration (“what this tool can do, what it cannot do, what side effects it produces”), and every tool invocation should be recorded in an auditable trace. The MCP protocol [?] is evolving in this direction.
- **L5—Orchestration:** permission approval and failure recovery. High-risk operations (writing files, sending emails, executing shell commands) must pass through the deterministic control plane’s approval workflow. Failure recovery must have explicit rollback strategies (e.g., git revert) and human-takeover mechanisms.
- **L6—Application:** intent confirmation and side-effect declarations. Before executing any irreversible operation, the system should confirm with the user that its understanding is correct. After each user task completes, the system should generate a side-effect report (“which files were modified, which emails were sent, which external services were accessed”).

9.5.4 Governance in the Dual-Plane Architecture

Viewed through the lens of the dual-plane architecture (Section ??), governance is the *central component* of the deterministic control plane. Its role is not to replace the probabilistic plane’s reasoning capability but to set boundaries, maintain audit trails, and provide rollback mechanisms for the results of that reasoning. Concretely: the probabilistic plane is responsible for “understanding user intent, decomposing tasks, and generating tool-call parameters”—the reasoning capabilities at which LLMs excel; the deterministic plane is responsible for “checking whether permissions are satisfied, whether tool calls are safe, whether results are compliant, and whether failures are recoverable”—the deterministic logic at which traditional software engineering excels. CaMeL’s capability manager [?], Codex’s approval policies [?], and Claude Code’s hooks [?] are all concrete implementations of this principle.

This division of labor mirrors a classic design philosophy in operating systems: “the kernel does not compute on behalf of user programs, but ensures that their computation does not exceed its bounds” [?]. The kernel does not sort your array, but it ensures your program cannot read another process’s memory. Similarly, the governance layer does not perform reasoning on behalf of the agent, but it ensures that the agent’s reasoning outputs do not lead to unauthorized operations, data exfiltration, or unrecoverable failures.

9.5.5 The Formalization Challenge

Embedding governance into the architecture still faces a fundamental formalization challenge: how do we define “security boundaries” for probabilistic behavior? Traditional OS security models rest on deterministic state machines: every system call has well-defined preconditions (e.g., “the file descriptor must be valid”) and postconditions (e.g., “returns the number of bytes read”). Agent behavior, however, is probabilistic: the same operation may produce different results in different contexts, and whether a result is “safe” often depends on semantic judgment rather than formal verification. ArbiterOS’s policy specification [?] and IronClaw’s behavior-space modeling [?] represent first steps in this direction, but a practical formal governance framework remains distant. This is precisely the critical research direction that ICAM and the dual-plane architecture delineate for future work.

Within ICAM, governance is not localized to a single layer but is a *cross-cutting concern* that manifests differently at each level of the stack—from hardware precision guarantees (L1) to user-facing intent confirmation (L6). The dual-plane architecture provides the structural mechanism by which governance invariants are enforced: the deterministic control plane acts as the architectural “kernel” that mediates all interactions between the probabilistic reasoning engine and the external world, ensuring that Axiom 6 (“every irreversible action must be approved by the deterministic control plane”) is upheld regardless of which ICAM layer originates the action. This perspective transforms governance from an operational afterthought into an architectural invariant—a first-class design principle rather than a retrofit.

10 Open-Source Implementation and Engineering Practice

Viewed through an engineering lens, today’s open-source ecosystem already contains several projects that serve as de facto prototypes for the model-native computing architecture envisioned in this paper. In this section, we survey representative systems along three dimensions of the ICAM stack—the serving layer (L2), the agent layer (L5), and the protocol layer (L4)—and distill five concrete implementation recommendations grounded in both industrial practice and the design axioms introduced earlier.

10.1 Serving Layer (L2): Inference Serving and KV Cache Management

The serving layer corresponds to ICAM’s L2 and is responsible for model weight loading, KV cache management, continuous batching, and prefill–decode scheduling. This layer has reached a high degree of engineering maturity, with three distinct design philosophies emerging in production systems.

vLLM centers on PagedAttention, which organizes the KV cache into fixed-size blocks and maps them to non-contiguous physical GPU memory through a block table, effectively replicating the virtual memory management of a conventional operating system [?, ?]. Its Automatic Prefix Caching feature extends this analogy by enabling cross-request reuse of shared prompt prefixes—functionally equivalent to shared read-only code pages in an L2 hardware cache [?].

SGLang takes a complementary approach built around structured program execution and RadixAttention [?, ?]. RadixAttention maintains a radix tree that indexes the token prefixes of all historical requests, automatically discovering and reusing the longest common prefix KV cache. This yields notably higher cache hit rates in multi-turn dialogue and batch inference workloads. Empirical evaluations show that SGLang achieves up to 29% higher throughput than vLLM on certain workloads [?].

TGI (Text Generation Inference) from Hugging Face and **TensorRT-LLM** from NVIDIA

represent production-oriented deployment paths [?, ?]. TGI emphasizes a unified multi-model interface and operational simplicity, while TensorRT-LLM pursues maximum throughput through hardware-specific optimizations such as FP8 quantization and Tensor Core acceleration on NVIDIA GPUs.

Together, these three families of systems—vLLM (virtual-memory-style management), SGLang (structured cache reuse), and TGI/TensorRT-LLM (hardware-deep optimization)—constitute the full engineering spectrum of the L2 serving layer.

10.2 Agent Layer (L5): Agent Runtimes and Operating Systems

The agent layer maps to ICAM’s L5 and is responsible for task planning, agent scheduling, permission approval, failure recovery, and state commitment. This is the most active and diverse area of the current open-source ecosystem, spanning three sub-directions.

10.2.1 General-Purpose Agents

OpenHands provides an open agent platform supporting multiple LLM backends, sandboxed execution, and extensible tool interfaces [?]. **SWE-agent** targets software engineering tasks, unifying code editing, testing, and debugging through a structured Agent-Computer Interface (ACI) [?]. **AutoGPT** explores autonomous, long-running workflows with automatic goal decomposition [?]. **Letta** (built by the original MemGPT team) focuses on stateful agents that treat long-term memory as a first-class citizen [?].

10.2.2 Agent Operating Systems

Several projects have begun to treat the agent runtime itself as an operating system. **ArbiterOS** introduces a governance layer and proposes a hierarchical architecture in which a governor manages the “probabilistic CPU” [?]. **AgentOS** constrains agent behavior through a reasoning kernel governed by structured OS logic [?]. **UFO**² demonstrates the feasibility of a desktop-scale agent OS, organizing a HostAgent, AppAgents, and a GUI-API operation layer into a hierarchy [?]. **Aura** brings a security-first agent architecture to mobile devices, comprising a System Agent, sandboxed App Agents, and an Agent Kernel [?].

10.2.3 Industrial Agent Systems

Two industrial systems merit particular attention because they explicitly adopt OS-building conventions. **OpenAI Codex** elevates sub-agents, skills, MCP server integration, sandboxed execution, and approval policies to first-class citizens in its runtime [?, ?, ?, ?, ?]. **Anthropic Claude Code** offers sub-agents, skills, hooks, MCP connections, and project memory files, defaulting to read-only permissions with explicit approval gates [?, ?]. Both systems signal that the industry has begun to construct agent runtimes as bona fide operating systems.

10.3 Protocol Layer (L4): Standardized Tool Connection and Agent Interconnection

The protocol layer corresponds to ICAM’s L4 and is undergoing a fundamental transition from point-to-point tool invocations to standardized bus protocols.

MCP (Model Context Protocol) provides a standardized protocol for connecting AI applications to external data sources, tools, and workflows through a JSON-RPC specification [?]. By the end of 2025, Anthropic reported over 10,000 active public MCP servers [?]. In the analogy

to traditional computer architecture, MCP plays the role of a peripheral bus such as PCIe or USB—a universal connector between the compute core and external capabilities.

A2A (Agent-to-Agent Protocol), introduced by Google in April 2025, targets inter-agent communication as a standardized protocol supporting capability discovery, task lifecycle management, and multimodal collaboration [?, ?]. It has been donated to the Linux Foundation for vendor-neutral governance [?]. If MCP is the *vertical bus* connecting an agent to its tools, A2A is the *horizontal interconnect* linking agents to one another—analogue to TCP/IP in the traditional network stack. Together, MCP and A2A form complementary axes of the protocol layer.

10.4 Five Implementation Recommendations

Drawing on the engineering practices surveyed above and the six design axioms proposed in this paper, we offer five recommendations that can directly guide the construction of production model-native computing systems.

10.4.1 Separate the Control Plane from the Data Plane

Deterministic control logic—permission approval, audit logging, failure recovery—must be peeled away from the probabilistic inference path. These two paths correspond, respectively, to the deterministic control plane and the probabilistic execution plane in the dual-plane architecture introduced in Section ???. Codex’s approval policies and Claude Code’s hooks are concrete instantiations of this principle [?, ?].

10.4.2 Separate Short-Term Context from Long-Term Memory

High-frequency “hot context” (the active dialogue window) and low-frequency “cold memory” (historical summaries, knowledge-base indices) should reside on different storage media with different management strategies. This maps directly to the hot/warm/cold tiering design of ICAM’s L3 layer. MemGPT’s virtual context management exemplifies this principle [?]. The business impact is tangible: Augment Code reported that a project a CTO estimated would take 4–8 months was completed in just 2 weeks, demonstrating how effective context management accelerates developer onboarding and project delivery in enterprise settings [?].

10.4.3 Version Every Behavior-Affecting Object

Prompt templates, tool schemas, skill packages, memory files, and MCP server capability descriptions should all carry version numbers and changelogs [?, ?]. Traditional computers guarantee backward compatibility through the stability of ISA and ABI boundaries; model-native computing systems require an analogous “interface contract” to accommodate rapid evolution.

10.4.4 Enforce Resource Quotas Like an Operating System

Every agent or task should operate under an explicit resource budget: an upper bound on context window size, tool call count, inference time, and monetary cost [?]. This recommendation instantiates Axiom 5 (Least Privilege) and Axiom 6 (Observability) at the resource management layer, mirroring the role of cgroups in Linux.

10.4.5 Design Failure Recovery Like a Distributed System

Long-running agent tasks may fail due to model timeouts, tool errors, or context overflow. The system must provide checkpoint, retry, and rollback mechanisms—akin to those in distributed databases—ensuring that side-effect operations remain auditable and reversible [?, ?]. Rakuten’s experience with Claude Code provides compelling evidence: an agent autonomously completed an activation-vector extraction task across 12.5 million lines of code in the vLLM project over 7 hours with 99.9% numerical accuracy, demonstrating that long-running agents with implicit checkpoint and retry capabilities can already handle industrial-scale workloads [?].

11 Research Roadmap

Table ?? presents a research roadmap organized around the ICAM six-layer model, spanning three phases: short-term (1–2 years), mid-term (2–4 years), and long-term (4–8 years). Each entry specifies the problem to be solved, the proposed method, and the expected outcome, making every item a concrete, actionable research project.

Table 5: Research agenda organized by ICAM layer, phase, and associated law or axiom.

Phase	ICAM Layer	Topic	Method	Metrics	Related Law / Axiom
Short-term	L2	KV cache hit-rate optimization	Semantic eviction strategies; KV quantization and compression; prefix sharing	H , S , TTFT	Semantic Law Locality
Short-term	L3	Unified context compiler	Hot / warm / cold tiering with version stamps and TTL annotations	W_{eff} , compression ratio	Context Law Budget
Short-term	L4	Tool ABI standardization	Schema semantic versioning; Capability annotation	Invocation success rate; security violation rate	Axiom 5 (Least Privilege)
Short-term	L5	Software engineering agent kernel	Unified ACI specification; specialized sub-agent decomposition	Task completion rate; E	Agent Law Speedup
Mid-term	L3	Semantic page replacement	Learned eviction policies; summary write-back and retrieval	$\beta(L)$ curve	Context Law Budget
Mid-term	L5	Consistency-controlled shared memory	Event sourcing; conflict detection	Memory correctness rate	Axiom 4 (Virtualization)
Mid-term	L2	Heterogeneous model collaboration	Draft models (Speculative Decoding); MoE routing	End-to-end latency; H	Semantic Law Locality
Mid-term	L4	Agent security architecture	Sandboxing; auditing; Capability security	Audit completeness	Axioms 5, 6
Long-term	L1–L6	Intelligent ISA and programmable skill layer	Task DSL; controlled output formats; reusable skill packages	Generalization ability	Axioms 2, 3
Long-term	L3–L5	Stateful individual agents	Long-term memory; experience abstraction and policy learning	Experience utilization rate	Context Law, Agent Speedup Law

Phase	ICAM Layer	Topic	Method	Metrics	Related Law / Axiom
Long-term	L1–L6	Distributed model-native compute fabric	Memory replication; consistency layers; load balancing	Cluster utilization	Semantic Locality Law, Agent Speedup Law
Long-term	L5	Governance-layer primitives	Dual-plane separation; audit logs; permission declarations	Governance coverage	Axioms 3, 5, 6

11.1 Short-Term Research Topics (1–2 Years)

The short-term items focus on optimizing and standardizing existing engineering prototypes, with the goal of bringing current systems to production-grade performance and reliability.

KV cache hit-rate optimization (L2). The decoding phase of LLM inference is a classic memory-bandwidth-bound problem [?], and the KV cache hit rate H directly governs the inference speedup ratio S predicted by the Semantic Locality Law. Three complementary techniques can be brought to bear. First, *semantic eviction* strategies—exemplified by H2O’s heavy-hitter identification [?—retain the KV entries most likely to be reused based on attention weight patterns rather than recency alone. Second, *KV quantization* compresses the KV cache with minimal quality loss: KVQuant achieves up to $8\times$ compression [?], while KIVI introduces asymmetric 2-bit quantization [?]. Third, *prefix sharing* across requests—as implemented by PagedAttention [?] and RadixAttention [?—enables multiple requests with common system prompts to reuse the same KV blocks, directly increasing H . The target outcome is to raise H from the current 0.5–0.7 range to above 0.85 under typical agent workloads, yielding the $5\text{--}10\times$ speedup predicted by the Semantic Locality Law.

Unified context compiler (L3). Different agent frameworks manage context in idiosyncratic ways: MemGPT uses virtual context management [?], Letta builds stateful agents with explicit memory tiers [?], and HiAgent employs hierarchical working memory [?]. What is missing is a unified *context compiler*—analogous to an operating system’s page replacement subsystem—that decides, for each piece of information, whether to load it verbatim, as a summary, or as a retrievable index entry. We propose a hot / warm / cold tiered context management strategy, where each context block carries a version stamp and a time-to-live (TTL) annotation. The expected result is a $\geq 50\%$ increase in the effective context utilization ratio W_{eff}/C without degrading task completion rates, thereby providing direct empirical validation of the Context Budget Law.

Tool ABI standardization (L4). While MCP and A2A have made significant strides at the protocol level [?, ?], tool schemas themselves still lack unified specifications for versioning, backward compatibility, and capability annotation. We propose enriching tool schemas with semantic version numbers, input/output type constraints, side-effect declarations, and capability annotations—transforming L4 into a genuine Application Binary Interface (ABI) for the model-native computing stack. The expected outcome is a measurable increase in tool invocation success rates and a decrease in security violations, providing the interface foundation required to operationalize Axiom 5 (Least Privilege).

Software engineering agent kernel (L5). SWE-bench and SWE-agent have demonstrated that current code agents still achieve limited resolution rates on real-world GitHub issues [?, ?]. A key bottleneck is the lack of a unified Agent-Computer Interface (ACI) standard: different frameworks adopt inconsistent sub-agent delegation strategies. We propose standardizing the ACI specification and designing specialized sub-agents—a code-search agent, an edit agent, and a test agent—whose orchestration efficiency E can be quantified through the Agent

Speedup Law. The target is to increase task completion rates on SWE-bench-class benchmarks while raising E from the current 0.3–0.5 range to above 0.6.

11.2 Mid-Term Research Topics (2–4 Years)

The mid-term items shift focus from per-layer optimization to cross-layer coordination and system-level security, moving model-native computing from “point optimizations” to “full-stack co-design.”

Semantic page replacement (L3). Classical operating systems replace pages using deterministic access patterns (LRU, CLOCK). In a model-native computing system, “page replacement” must be governed by *semantic relevance*—deciding which context blocks to evict based on their pertinence to the current task. We propose a learned eviction policy that scores each context block against the active task and generates compressed summaries for evicted blocks, enabling later retrieval. The primary research output is an empirically measured $\beta(L)$ curve that characterizes how attention retention decays with position, providing the Context Budget Law with calibrated quantitative parameters.

Consistency-controlled shared memory (L5). When multiple agents collaborate, shared memory consistency management lacks a unified framework. Concurrent modifications to the same memory block can cause conflicts and information loss—the analog of cache coherence problems in multiprocessor systems. We propose introducing event sourcing and conflict detection mechanisms, combined with tunable consistency levels ranging from eventual consistency to strong consistency [?, ?]. The target is to ensure that multi-agent memory correctness rates match single-agent baselines, providing empirical grounding for Axiom 4 (Virtualization) at the memory layer.

Heterogeneous model collaboration (L2). Models of different sizes offer distinct trade-offs among latency, cost, and capability, yet current systems typically deploy a single model. Heterogeneous model collaboration—using smaller draft models for speculative decoding [?] and mixture-of-experts (MoE) routing [?, ?]—can direct simple queries to lightweight models and complex queries to large ones. The expected result is a reduction in end-to-end latency at equivalent output quality, with a secondary benefit of increased KV cache hit rates H .

Agent security architecture (L4). As agents acquire increasingly powerful external capabilities—reading and writing files, executing commands, accessing the network—the security perimeter becomes dangerously blurred. We propose embedding sandboxing, auditing, and capability-based security directly into the agent runtime [?, ?, ?], ensuring that every side-effecting operation undergoes explicit approval and produces an auditable event record. The target is 100% audit completeness—every side-effecting operation across all ICAM layers must be traceable—establishing the security infrastructure required to enforce Axioms 5 and 6 in practice.

11.3 Long-Term Research Topics (4–8 Years)

The long-term items target fundamental architectural innovations, with the goal of constructing a truly native model-based computing architecture rather than incrementally improving today’s systems.

Intelligent ISA and programmable skill layer (L1–L6). Prompts, tool descriptions, and skill packages collectively form the “soft instructions” of model-native computing, yet they currently lack formal semantics and stability guarantees. We envision developing task-specific domain-specific languages (DSLs), controlled output formats, and reusable skill packages that together constitute a stable interface layer—the model-native analog of a classical Instruction

Set Architecture. The expected outcome is significantly improved agent generalization, enabling the same skill package to migrate seamlessly across different models and runtimes, thereby operationalizing Axioms 2 (Layered Abstraction) and 3 (Probabilistic Execution).

Stateful individual agents (L3–L5). Most agents today are stateless: each session starts from scratch, unable to accumulate or reuse experience. We propose equipping agents with long-term memory and experience abstraction mechanisms, enabling them to learn strategies from historical executions, distill recurring patterns, and proactively optimize their own behavior [?, ?]. The expected outcome is a measurable increase in *experience utilization*—the fraction of current task performance attributable to distilled prior experience—which opens new validation dimensions for both the Context Budget Law and the Agent Speedup Law.

Distributed model-native compute fabric (L1–L6). When multiple agent clusters collaborate across nodes, distributed systems problems—memory replication, state consistency, and load balancing—remain largely unsolved in the model-native computing context. Drawing on distributed database techniques for memory replication and consistency management [?, ?], we propose a unified resource scheduling and state management substrate for multi-node agent clusters. The target is for cluster utilization to scale linearly (or at least near-linearly) with node count, empirically validating the Semantic Locality Law and the Agent Speedup Law in distributed settings.

Governance-layer primitives (L5). Current governance mechanisms in agent systems—permissions, auditing, compliance—are retrofitted patches rather than first-class architectural primitives. We propose elevating the dual-plane separation (probabilistic execution plane and deterministic control plane), audit logging, and permission declarations to core architectural primitives [?, ?], ensuring that every ICAM layer has a corresponding governance mechanism. The target is 100% governance coverage: every side-effecting operation at every layer falls under governance constraints, fully realizing Axioms 3, 5, and 6 across the entire stack.

11.4 A Note on Decoupling from Model Advances

It is worth emphasizing that these research topics do not require stronger foundation models as a prerequisite. Many of the problems—context compilation, Tool ABI standardization, agent security architecture—are fundamentally systems engineering challenges that are orthogonal to improvements in base model capability. This decoupling mirrors the history of classical computing: architectural breakthroughs have come not only from faster transistors but equally from more mature interfaces and runtimes. The stability of the ISA enabled a thriving software ecosystem; the invention of virtual memory freed programs from physical memory constraints; the POSIX standard allowed applications to migrate across platforms. Model-native computing systems stand in need of precisely the same kind of systems-level breakthroughs.

12 Ethics, Safety, and Explainability

Treating large-model systems as computers does not make ethical concerns disappear. If anything, the architectural lens renders certain ethical questions sharper, because each concern maps to a specific ICAM layer and its corresponding governance mechanism.

12.1 Permission and Isolation (L5 Orchestration Layer)

If agents are the new processes, then the questions of what privileges they hold, what secrets they can observe, whether they may access the network, and whether they may mutate persistent state must be answered through explicit security policies—analogueous to how an operating system

mediates process permissions [?, ?, ?, ?, ?, ?].

Concretely, the governance mechanisms at ICAM’s L5 layer must address four questions:

- **Who may create agents?** This is the analog of process-creation privileges. OpenAI Codex requires developers to declare agent behavioral boundaries through an explicit `AGENTS.md` manifest [?].
- **What resources may an agent access?** This corresponds to file-system permission bits. Claude Code adopts a default-read-only posture: every write operation requires explicit user approval [?].
- **How are permissions scoped to individual operations?** This mirrors capability-based security. CaMeL [?] attaches *capability tokens* to each tool invocation, ensuring that an agent can exercise only the rights explicitly granted for that particular call.
- **How is a misbehaving agent terminated?** This is the analog of process signals and sandbox isolation. Codex employs OS-level sandboxing so that all agent write operations are confined to a controlled environment [?]. IronClaw [?] extends this model by proposing hardware-enforced isolation boundaries for agentic workloads, drawing a direct parallel to hypervisor-based virtual machine isolation.

Viewed through the lens of the *dual-plane architecture* introduced in Section ??, these permission controls fall squarely within the remit of the *deterministic control plane*: the *probabilistic execution plane* determines what the model *can* do (reasoning and generation), while the deterministic control plane determines what it *is allowed* to do (approval and isolation).

12.2 Privacy and Memory Governance (L3 Memory Layer)

Privacy concerns concentrate at L3, the memory layer. Once a system supports long-term state, it must confront four governance questions—questions that have analogs in classical memory management but become substantially more complex in a semantic system.

What should be persisted? Not every interaction deserves long-term retention. Classical operating systems use page-protection bits to distinguish readable, writable, and executable pages; L3 must similarly distinguish between *persistable* information and *session-scoped* information. A user’s code-editing preferences may be safely persisted, whereas temporary credentials or sensitive conversation fragments should be tagged as session-scoped and automatically purged when the session ends.

For how long? This raises the question of retention policies. Traditional file systems manage temporary files through Time-To-Live (TTL) metadata; L3 must assign a TTL and a version number to every memory block. The design is further complicated by a direct policy conflict: the EU AI Act [?] requires high-risk AI systems to retain operational logs for at least ten years, while the GDPR’s Right to Be Forgotten [?] empowers users to request the deletion of personal data. Reconciling these obligations is not a matter of model design—it is a governance decision that L3’s retention-policy designers must address explicitly.

Who has the right to delete? In a multi-agent setting with shared memory, a memory block written by one agent may be referenced by others. Deletion cannot proceed naively; it requires a reference-tracking mechanism akin to garbage collection—a block may be physically freed only when all referencing agents have consented or have themselves terminated. This extends the Observability Axiom (Axiom 6): the act of deletion must itself be auditable.

Can summaries be reverse-engineered to reveal sensitive information? L3’s context compiler routinely compresses raw inputs into summaries to conserve context-window capacity. However, summaries may retain enough semantic signal that sensitive attributes—personal identities, financial data—can be inferred. This demands that L3’s semantic address contracts enforce four properties: *minimal retention* (store only task-essential information), *cascading deletion* (when raw data is removed, all derived summaries are purged together), *access isolation* (memory belonging to different users must not be cross-accessible), and *encrypted storage* [?, ?].

12.3 Explainability: From Explaining Models to Explaining Systems (Full-Stack)

Traditional LLM explainability research focuses on interpreting a model’s internal representations—the roles of attention heads, the semantics of individual neurons, and so on. From ICAM’s perspective, however, the more operationally actionable unit of explanation for an agent system running on a complete model-native computing architecture is not found inside the model, but in the system’s behavioral trace. This insight is the very core of the Observability Axiom (Axiom 6).

ICAM decomposes explainability into the following actionable tiers, each aligned with a specific layer:

- **L2**—Cache hit/miss logs: which context fragments were reused, which were evicted, and what policy governed each eviction decision.
- **L3**—Context-compilation decisions: which information was loaded verbatim versus in summary form, the provenance of each summary, and the compression ratio achieved.
- **L4**—Tool-call parameters and results: which tool was invoked, with what arguments, what it returned, and whether a permission denial was triggered.
- **L5**—Approval decisions and state commits: which agent requested which privilege at what time, whether the request was granted or denied, and what state mutation was ultimately committed.

This notion of *system-level behavioral-trace explainability* is more directly useful than model-internal explainability. It does not require prying open the model’s black box; instead, it demands a faithful record of what the system did at every step, why it did so, and what the outcome was. Viewed through the dual-plane architecture, this is precisely the core output of the deterministic control plane—an auditable, replayable event stream.

12.4 Fairness and Resource Allocation (L2 Inference Layer and L5 Orchestration Layer)

When multiple users or multiple agents share an inference-serving cluster, fair resource allocation becomes an ethical imperative. Classical operating systems enforce fair sharing of CPU, memory, and I/O through control groups (cgroups) [?]; model-native computing systems must analogously enforce per-agent or per-task budgets for context-window capacity, inference time, and monetary cost.

The specific fairness concerns include:

- **Inference-resource fairness.** Are GPU resources distributed equitably, or do high-volume users monopolize capacity?
- **KV-cache contention.** Do long-running agents hoard KV-cache entries, starving short tasks of memory—an analog of the classical memory-hog problem?
- **Side-effect equity.** When tool invocations produce side effects (e.g., modifying shared state), are those effects applied uniformly for all users, or do scheduling biases advantage

some agents over others?

These fairness questions are a direct extension of the Least-Privilege Axiom (Axiom 5) into the resource-management plane: an agent should consume no more than its fair share of shared resources, and the system must enforce this bound as a first-class invariant. The Anthropic 2026 agentic safety report [?] further highlights the dual-use risk: the same architectural mechanisms that enable capable autonomous agents can be repurposed for harmful ends, making robust permission isolation and resource governance not merely a fairness concern but a societal imperative.

13 Conclusion and Outlook

The central contributions of this paper can be distilled into a unified framework, an architectural resolution, a set of computable laws, and a first-class treatment of governance.

13.1 Core Contributions

Unified framework: the ICAM six-layer model. We have proposed the Intelligent Computing Architecture Model (ICAM), which defines a model-native computing system as six functional layers (L1–L6), each equipped with explicit interface contracts, design axioms, and performance metrics. ICAM is not a mechanical transcription of classical computer architecture; rather, it is a layered abstraction purpose-built for probabilistic execution. Its primary value lies in unifying previously disparate efforts—from AIOS [?] and MemGPT [?] to MCP [?] and PagedAttention [?—under a single, shared vocabulary that makes their interdependencies visible and their design trade-offs explicit.

Architectural resolution: the dual-plane architecture. We have identified the root cause of the enduring “Is an LLM a CPU or an OS?” metaphor clash and resolved it through the dual-plane architecture. The *probabilistic execution plane* (model inference) captures what the system *can* do; the *deterministic control plane* (schedulers, approvers, audit logs within the agent runtime) governs what the system *should* do. This separation reconciles ArbiterOS’s “Probabilistic CPU” [?], AIOS’s “kernel” [?], and AgentOS’s “reasoning kernel” [?]: they are not contradictory claims about a single component but complementary perspectives on two distinct planes within the same system.

Computable principles: three quantitative laws. The Semantic Locality Law ($S = 1/((1 - H) + H \cdot \alpha^{-1})$), the Context Budget Law ($W_{\text{eff}} \leq C \cdot \beta(L)$), and the Agent Speedup Law ($S_{\text{agent}} = 1/((1 - F) + F/(N \cdot E))$) provide model-native computing with the same kind of computable guardrails that Amdahl’s Law and the Roofline model have long offered classical architecture. These laws are isomorphic to Amdahl’s Law in form; their contribution is not mathematical novelty but rather the provision of a *quantitative decision language* for a design space that has thus far relied primarily on intuition. Empirical validation against published experimental data shows qualitative agreement, though systematic quantitative verification remains an important next step.

Governance as a first-class concern. Unlike prior architectural proposals that treat safety and auditability as afterthoughts, our framework elevates governance—permission boundaries, approval workflows, and verifiable audit trails—to a structural element woven into every layer from L3 through L6.

13.2 Positioning and Boundaries

We explicitly acknowledge the following limitations.

Analogies are not isomorphisms. A foundation model is not a deterministic CPU, semantic retrieval is not exact addressing, long-term memory is not lossless paging, and an agent runtime is not a mature operating system. Yet these very gaps—the places where the analogy breaks down—define the most fertile ground for future research: new ABIs for semantic systems, new operating system abstractions for uncertain reasoning, new virtual memory designs for persistent state, and new commit protocols for multi-agent coordination.

Incremental, not inaugural. This paper does not claim to be the first to draw parallels between large language models and computer architecture. Ge et al. [?], L2MAC [?], and Mi et al. [?] have mapped several key routes, and MemGPT [?] and MemOS [?] have already systematized memory subsystems to a high degree. Our contribution lies in being the *first to integrate these independently developed—and sometimes mutually conflicting—analogies into a full-stack model-native computing architecture*, complete with a unified layered model, a dual-plane architecture, inter-layer interface contracts, an axiom system, and computable quantitative laws.

Visionary survey, not final answer. This paper is a visionary survey that aims to provide a research framework for future model-native computing architectures, not a definitive specification. The six-layer decomposition of ICAM and the parameterized forms of the three laws will almost certainly be revised and refined as the field matures. Our goal is to pose the right questions and offer falsifiable predictions, not to deliver the last word.

The collaboration paradox remains uncaptured. Our framework does not yet fully model the structural constraints of human–AI collaboration. Industry evidence reveals that engineers use AI for approximately 60% of their work yet can fully delegate only 0–20% of tasks [?]. This “collaboration paradox” suggests that the task-submission contracts at the L5–L6 interface must incorporate a *delegation confidence* dimension—reflecting task verifiability and organizational context dependence—that lies beyond the current ICAM model’s scope.

13.3 Future Directions

If the past decade focused on training stronger foundation models, the next decade’s central question may be: how to organize foundation models, contextual memory, tool interfaces, and agent runtimes into a truly programmable, scalable, and governable model-native computing architecture.

The answer is likely to emerge from the convergence of advances across multiple fronts simultaneously:

- At L2, inference serving optimizations—KV cache management, quantization, speculative decoding—are making model serving ever more efficient.
- At L3, context compilation and memory management are beginning to give agents genuine working memory.
- At L4, protocol standardization (MCP, A2A) is shifting tool and agent interconnection from point-to-point integration to a bus-based paradigm.
- At L5, agent operating systems are moving task decomposition, permission control, and failure recovery from hand-coded scripts to platform-native primitives.
- At L6, workflow automation is enabling end users to define complex tasks in natural language.

The economic evidence is already compelling. TELUS has created over 13,000 customized AI solutions, saved more than 500,000 hours, and accelerated engineering code delivery by 30%.

CRED has doubled execution speed by shifting developers to higher-value work. Approximately 27% of AI-assisted tasks consist of work that would not have been done at all without AI [?]. These figures demonstrate that model-native computing does not merely accelerate existing work—it expands the frontier of economic feasibility.

This paper offers a systematic research framework for the road ahead: a layered model (ICAM), a set of design axioms, three computable laws, and a research roadmap spanning near-term optimizations to long-term architectural vision. We hope this framework will help researchers and engineers identify system-level questions worth asking and discover verifiable design principles amid the rapidly evolving model-native computing ecosystem.

Table 1: Analogy mapping between classical computer architecture and the model-native system stack. Each row pairs a classical concept with its model-native counterpart, along with the core similarity, fundamental difference, and resulting engineering implication.

Classical Concept	Model-Native Counterpart	Core Similarity & Example	Fundamental Difference	Engineering Implication
ISA / ABI (Application Binary Interface)	Prompt templates, tool specifications, Skill/Agent interfaces	Both define the calling contract and composition boundary visible to upper layers. Example: RISC-V’s ECALL instruction has precisely defined semantics[?]; MCP’s tools/call interface specifies parameter schemas and return formats[?]	Prompts lack strict formal semantics—ambiguity and drift are possible; the same prompt may behave differently across model versions	Requires stable schemas, version identifiers, and replay compatibility
CPU Core	Foundation model forward pass	Both are general-purpose execution cores that map inputs to outputs. Example: a CPU maps instruction sequences to register state changes; a model maps token sequences to next-token probability distributions[?, ?]	The model performs probabilistic inference, not deterministic logic; identical inputs can yield different outputs	Explicitly decouple model capability, inference strategy, and runtime
Microarchitecture	Attention kernels, decoding algorithms, serving engines	Both determine the real performance and cost behind the same “interface.” Example: superscalar pipelines vs. FlashAttention’s I/O-aware tiled computation [?]; branch prediction vs. speculative decoding[?]	Model-era “microarchitecture” spans the software/hardware boundary	Joint optimization of FlashAttention, speculative decoding, and parallelism strategies
L1/L2 Cache	Session-level KV cache, prefix caching	Both exploit temporal locality and hotspot reuse. Example: a CPU L1 cache retains recently accessed cache lines; a KV cache retains recently computed Key/Value vectors to avoid redundant computation[?, ?]	Semantic cache hit criteria are far weaker than address-based hit criteria; address equality is a Boolean check, while semantic similarity is a continuous-valued comparison	Requires block-level allocation, sharing, invalidation, and monitoring
Main Memory / Virtual Memory	Context window, external memory, virtual context	Both create the illusion of a large space through layering. Example: virtual memory gives each process the illusion of a contiguous, large address space; virtual context management gives an agent the illusion of unlimited memory[?]	Semantic summarization loses information—it is not lossless paging; OS page swap is byte-exact, while summary compression is irreversible	Requires hot/warm/cold memory tiering, re-retrieval, and backfill strategies
Operating System	Agent runtime	Both manage scheduling, permissions, I/O, and failure handling. Example: Linux cgroups isolate process resources[?]; Codex	Agents entail non-deterministic reasoning and language-mediated interfaces	Requires sandboxing, approval gates, sub-agent orchestration, hooks, and logging

Table 2: ICAM layers mapped to classical computer architecture.

ICAM Layer	Classical Analog	Responsibility	Core Difference	Representative Implementations
L1	Hardware / ISA	Execute basic compute operations	Probabilistic matrix operations, not deterministic logic gates	GPU, TPU, Compute-in-Memory[?]
L2	Microarchitecture / Cache	Manage compute resources and hot data	KV cache grows with semantic sequence, not address-indexed	vLLM, SGLang[?, ?]
L3	Virtual Memory	Expand the addressable working set	Semantic summarization is lossy, not lossless paging	MemGPT, MemoryOS[?, ?]
L4	Syscall / ABI	Uniform interface to external capabilities	Tool semantics carry natural-language ambiguity	MCP, A2A[?, ?]
L5	Operating System	Scheduling, permissions, fault tolerance	Agents themselves contain non-deterministic reasoning	Codex, Claude Code[?, ?]
L6	User Application	Carry end-user tasks	Tasks may have open-ended goals and fuzzy constraints	SWE-agent, OpenHands[?, ?]

Table 3: Cross-mapping of the dual-plane architecture onto ICAM layers.

ICAM Layer	Probabilistic Execution Plane	Deterministic Control Plane
L1	Matrix operations, attention computation (probabilistic output)	Precision verification, hardware error detection
L2	Semantic KV-cache matching, prefix sharing	Cache capacity management, eviction policy
L3	Semantic retrieval, context compilation, summarization	Memory retention policy, version stamps, state persistence
L4	Semantic understanding of tool results	Tool schemas, protocol specifications, permission enforcement, auditing
L5	Agent reasoning and decision-making (probabilistic)	Task scheduling, access control, failure recovery, trace logging
L6	Consumer of both planes: users receive intelligent output <i>and</i> an auditable trace	

Table 4: Empirical evidence for the three quantitative laws.

	Law I (Semantic Locality)	Law II (Context Budget)	Law III (Agent Speedup)
Core formula	$S = 1/((1 - H) + H/\alpha)$	$W_{\text{eff}} \leq C \cdot \beta(L)$	$S = 1/((1 - F) + F/(NE))$
Classical analogue	Amdahl’s Law ($f \leftrightarrow H$, $p \leftrightarrow \alpha$)	Denning working-set model ($W(\tau) \leftrightarrow W_{\text{eff}}$)	Amdahl + orchestration overhead ($E < 1$)
Supporting data	vLLM: 2–4× throughput gain ($H \approx 0.56\text{--}0.83$) [?]; Prompt Cache: 8–60× TTFT reduction ($H \approx 0.75\text{--}0.98$) [?]	RULER: accuracy drops from >95% at 4K to <70% at 128K [?]; LongBench v2: faster decay for deep reasoning [?]	AutoGen: diminishing returns from 2 to 8 agents [?]; GTA-2: open-workflow success only 14.39% [?]
Open questions	Distribution of H under semantic caching	Precise functional form of $\beta(L)$	Quantitative relationship between E and task complexity